

PARALLAX-5: A Five-Obligation Substrate for Smart Contracts and AI Agents

Benjamin P. Duncan
AquaUrsa Research

<https://github.com/aquaursa/parallax-5>

Paper DOI: 10.5281/zenodo.20402755 (v1.0.1)

Original v1.0.0 DOI: 10.5281/zenodo.20400525

Verification artifact: doi:10.5281/zenodo.20386868

May 2026

Paper licensed under CC-BY 4.0. Standard text under CC0 1.0. Code under Apache-2.0.

Abstract

We propose PARALLAX-5, a transition-level obligation interface for value-bearing decentralized systems. The interface consists of five primitive obligations: value conservation, authorization closure, signature integrity, temporal distinctness, and external-attestation trust. We prove that, under an explicit security-interface adequacy condition, every trust-base-respecting loss-inducing transition has a non-empty violation signature; the claim is falsifiable by basis counterexamples and we define them precisely. We introduce a parameterized basis-observability predicate $\text{ObservableUnder}_\Omega(t)$ relative to four canonical observation sets $\Omega_{\text{chain}} \subset \Omega_{\text{config}} \subset \Omega_{\text{intent}} \subset \Omega_{\text{infra}}$, making prior basis-observable classifications principled rather than binary. We define a step-secure gate that checks both post-state invariants and transition obligations and prove it is the maximally permissive accept/reject shield preserving the basis. Monitor soundness alone suffices for safety. The same gate composes with adaptive agent policies for session-level guarantees, with cross-VM refinement for transfer across domain machines, and with an on-chain indistinguishability result that identifies the irreducible scope boundary. The Lean 4 module compiles to 95 theorems with zero `sorry`; all 129 Python fire tests pass. Empirically, our 53-incident catalog (2016–2026, \$5.97 billion aggregate losses) classifies each entry by its minimum observability set: \$3.60B at Ω_{chain} , \$1.37B additionally at Ω_{config} , and \$1.00B genuinely irreducible.

The substrate is composed with a production EVM semantics in Lean 4 via a typeclass-based refinement: nineteen abstract theorems lift to compiled Lean 4 proof terms over Nethermind’s `EvmYul.EVM.State` (Cancun fork) under a single instance registration, with the verification artifact deposited at doi:10.5281/zenodo.20386868. The integration achieves interface transport and partial semantic fidelity (Section 18); contract-level verification of deployed bytecode is the proper subject of downstream tooling. A compositional verification architecture (Theorems 47, 48) treats the joint coverage of a tool stack as the pointwise maximum of per-tool capabilities. The substrate’s ecosystem layer adds a walk-away property formalizing founder-independent operation; a five-component PARALLAX-CROPS vector extending the single-axis depth scale; a 19-field machine-checkable certificate schema with seven-state lifecycle; and three worked examples covering value conservation, bridge attestation, and AI-agent runtime gating. The substrate’s standard text is published under CC0 with structurally irrevocable non-capturability commitments: no trademark on the name, no AquaUrsa patent claims over the disclosed techniques, no token, no governance body.

Contents

Reading Guide	4
Notation and Abbreviations	4
1 Introduction	5
2 Threat Model and Trust Base	7
2.1 Trust Base Assumptions	7
2.2 Transition-Level Model	7
2.3 Loss Predicate	7
3 The Five Obligations	7
3.1 A_1 Value Conservation	7
3.2 A_2 Authorization Closure	8
3.3 A_3 Signature Integrity	8
3.4 A_4 Temporal Distinctness	8
3.5 A_5 External-Attestation Trust Boundary	8
4 Conditional Completeness and Falsifiability	9
5 Minimality, Independence, and Closure	9
6 Step-Secure Gates	9
6.1 Step Security	9
6.2 The Maximal Safe Gate	9
6.3 Maximal Permissive Safety	10
6.4 Monitor Soundness Suffices	10
7 Adaptive AI-Agent Safety	10
8 Cross-VM Refinement	10
9 Off-Chain Trust Base and Indistinguishability	10
10 Empirical Catalog	11
10.1 Key Reclassifications	11
10.2 Bytecode-Level Reproduction	12
10.3 Exploit Normal Form	12
11 Economic Security	12
11.1 The Per-Attack Theorem	12
11.2 Critical Adoption Rate with Monitor False Negatives	13
11.3 Loss-Prevention Ceiling with the Two-Axis Model	13
11.4 Parameterized Basis-Observability	13
11.5 Unified loss ledger	15
12 The Walkaway Property	15
12.1 Definition and motivation	15
12.2 Five-level classification	16
12.3 Mechanized theorems	16
12.4 Relation to existing terminology	17

13 2026 Forward-Test	17
14 PARALLAX-5 as a Unification Layer	18
15 The PARALLAX-CROPS Trust-Surface Vector	18
15.1 The five dimensions	19
15.2 The contribution matrix	19
15.3 Projection function	20
15.4 Uniswap V3 Core	20
15.5 Consumer-side policy semantics	20
16 Worked Examples	20
16.1 The ERC-4626 inflation attack	21
16.2 Bridge attestation	21
16.3 An AI-agent runtime gate	22
16.4 What the demos jointly show	23
17 Comparison with Existing Standards	23
17.1 Standard Infrastructure	23
18 The EVM Backend	24
18.1 Choice of backend	24
18.2 The <code>EvmLikeMachine</code> typeclass	24
18.3 The refinement theorem and its instantiation	24
18.4 The instance over <code>EvmYul.EVM.State</code>	25
18.5 Verification protocol and reproducibility	27
18.6 Scope	27
19 Compositional Verification via Obligation Mapping	27
19.1 Problem statement	28
19.2 Coverage as a graded function	28
19.3 Two theorems	28
19.4 The TOOL-MAPPING calibration artifact	29
19.5 Two observations from the matrix	29
19.6 Empirical study: the discovery and held-out corpora	30
19.7 Case study: the Mango Markets compositional case	30
19.8 The coordinator artifact	30
19.9 Threats to validity	31
19.10 Implication for the PARALLAX-5 standard	31
20 Adoption Pathways	31
21 Tooling and Artifact	32
21.1 Theorem Inventory with Lean Names	32
21.2 Conditional ObligationSol Soundness	32
22 The Certificate Schema	33
22.1 Top-level fields	33
22.2 Lifecycle state machine	35
22.3 Canonical serialization and fingerprint	35
22.4 Evidence resolution	35
22.5 Registry events	36
22.6 Conformance requirements	36

23 Substrate Governance	37
23.1 The Non-Capturability Charter	37
23.2 License Layering	37
23.3 The Fork Protocol	38
23.4 Why CC0 is structurally necessary	38
24 Claims Table	39
25 Related Work	39
26 Limitations	40
27 Conclusion	40
A A Quantitative Risk Framework	43
B Codebook Inter-Rater Agreement	45
C Protocol Archetypes	46
D Artifact Map	47

Reading Guide

This paper is written for several audiences with different priorities. The structure supports non-linear reading.

- **Formal-methods reviewers** should read §3–9 (the formal core), §12 (walkaway), §18 (EVMYulLean composition, with attention to the explicit fidelity caveats in Table 11), and §19 (compositional architecture). The Claims Table in §24 separates theorems from assumptions from artifact claims.
- **DeFi security practitioners** should read §3 (the five obligations), §10 (the 53-incident catalog), §16 (three worked examples), and §22 (the certificate). The audit-to-certificate workflow lives in the case studies.
- **Ethereum-aligned infrastructure and policy audiences** should read §12 (walkaway), §15 (PARALLAX-CROPS), §16 (worked examples, especially Demo 3 on AI-agent containment), and §23 (non-capturability commitments).
- **Risk and underwriting audiences** should read §10 and the unified loss ledger in §11.5, then §A (the framework sketch; per-protocol calibration is deferred to a companion document).

The paper’s central technical claim is the conditional-completeness theorem (§4). The paper’s central practical artifact is the certificate (§22). The paper’s central commercial wedge is the AI-agent runtime gate (§16.3). Readers pressed for time can begin with whichever of these is most relevant and trace cross-references from there.

Notation and Abbreviations

Obligation symbols.

- A_1 value conservation; A_2 authorization closure; A_3 signature integrity; A_4 temporal distinctness; A_5 external-attestation trust.
- $\sigma(t) \subseteq \{A_1, \dots, A_5\}$: the violation signature of transition t — the set of obligations t violates.

Depth scale.

- $D_0, \dots, D_5$: the proof-depth scale. D_0 = no coverage; D_1 = comment-grade mention; D_2 = static-detector pattern match; D_3 = symbolic execution; D_4 = formal proof; D_5 = runtime enforcement.

Observation sets.

- $\Omega_{\text{chain}} \subset \Omega_{\text{config}} \subset \Omega_{\text{intent}} \subset \Omega_{\text{infra}}$: the four canonical observation sets a monitor may read (§11.4).
- $\text{ObservableUnder}_\Omega(t)$: the basis-observability predicate for transition t under observation set Ω .

Gate predicates.

- $\text{StateSecure}(s')$: the basis-respecting predicate over post-states.
- $B(t)$: the basis-respecting predicate over transitions.
- $\text{StepSecure}(t) := \text{StateSecure}(s') \wedge B(t)$: a step-secure gate accepts t iff $\text{StepSecure}(t)$ holds.

PARALLAX-CROPS dimensions.

- $(C, R, O, P, S) \in D^5$: the five-component trust-surface vector. C censorship-resistance; R capture-resistance (separate from C); O openness; P privacy; S aggregate security.

Walkaway classification.

- Full / Bounded / Partial / Centralized / Fake: the five-level classification (§12).
- $\text{Adm}(P)$: the set of off-chain parties holding a privileged role in protocol P .

Loss-ledger quantities (§11.5).

- $L_{\Omega_{\text{chain}}}, L_{\Omega_{\text{config}}}, L_{\Omega_{\text{intent}}}, L_{\text{irreducible}}$: losses rejectable by a sound monitor with the given observation set, partitioned exhaustively over the 53-incident catalog.

Frequently used abbreviations.

- TVL: total value locked; CDP: collateralized debt position; AMM: automated market maker; CPMM: constant-product market maker; ECDSA: Elliptic Curve Digital Signature Algorithm; EUF-CMA: existential unforgeability under chosen-message attack; SMT: satisfiability modulo theories.
- EVM: Ethereum Virtual Machine; EVMYulLean: Nethermind’s Lean 4 mechanization of the EVM, validated against the Ethereum test suite at the Cancun fork; ArkLib: an EF-funded zkEVM Lean library.
- CC0: Creative Commons Zero (public domain dedication); RFC: Request for Comments (specification draft).

1 Introduction

The central claim, stated precisely. The central claim of this paper is falsifiable. A *basis counterexample* is a trust-base-respecting transition that causes protected-value loss while satisfying all five obligations. Our formal results show that, under a stated adequacy condition, no such transition exists in the abstract model; our empirical results show that no such transition appears in our 41-incident basis-observable corpus, including off-chain-rooted attacks whose on-chain consequence violates the basis. We do not claim full semantic completeness over arbitrary EVM bytecode. The substrate is a compact security interface whose adequacy can be challenged, refined, and mechanically checked.

Theorem 2 is not a semantic-completeness theorem over all bytecode. It is a *decomposition theorem*: once the security interface is accepted as adequate for a transition class,

all loss-inducing transitions in that class have a non-empty violation signature. The substantive claims are exposed as explicit adequacy obligations and empirical falsification targets, not hidden inside the proof.

Motivation. Decentralized finance has lost approximately \$10 billion to exploits since 2020 [3, 4]. Despite extensive auditing and existing static analyzers, the same vulnerability patterns recur. The community has accumulated a large taxonomy of attack vectors [5, 6] but lacks a unifying theoretical framework that is simultaneously (a) small enough to fit on one page, (b) precise enough to be mechanically checkable, and (c) general enough to extend across execution environments and to AI-agent contexts.

Two-axis empirical classification. We classify each historical incident along two distinct axes. The *root-cause axis* records where the failure originated (on-chain code, off-chain key, off-chain signer, off-chain infrastructure, or mixed). The *basis-observability axis* records whether the malicious transition would be rejected by a sound gate. These axes are not aligned: off-chain root causes can produce *basis-observable* consequences (Resolv’s unbacked mint, Drift’s fake collateral whitelist, Kelp DAO’s invalid bridge message) that a sound gate would reject. The irreducible residual under on-chain enforcement is $L_{\text{basis-unobservable}}$, not the naive off-chain total.

Contributions. The paper’s contributions are:

- **Conditional completeness** (Theorem 4): under an explicit adequacy assumption, every trust-base-respecting loss-inducing transition violates at least one obligation. The assumption is itself falsifiable.
- **Falsification criterion** (Theorem 5): completeness over a transition class is equivalent to absence of basis counterexamples in that class.
- **Step-secure gate and gate transition safety** (Theorem 12): a gate that checks step security — state-level post-condition *and* transition predicate B — preserves not only post-state invariants but *also* the basis predicate of every executed transition. The naive post-state-only gate is insufficient because A_2, A_3, A_4 are transition properties not inferable from post-state alone.
- **Maximal permissive safety** (Theorem 13): among all safety-preserving gates, the step-secure gate G^* accepts the largest set of actions.
- **Adaptive session safety** (Theorem 15): for any agent policy, iterated G^* keeps the state in the invariant envelope.
- **Step-secure refinement transfer** (Theorem 18): with step-security preservation as the missing hypothesis, the abstract gate safety theorem genuinely commutes across the abstraction.
- **Mechanically verified EVMYulLean composition** (Theorem 43): nineteen abstract refinement theorems lift to compiled Lean 4 proof terms over Nethermind’s `EvmYul.EVM.State` (Cancun fork) under a single typeclass instance registration; the verification is byte-level reproducible across three independent runs and externally re-verifiable via the deposit at `doi:10.5281/zenodo.20386868` [1].
- **Compositional verification architecture** (Theorems 47, 48): a graded-evidence model under which the joint coverage of a tool set is the pointwise max of per-tool capabilities, and adding a tool to the stack never lowers the certifiable compliance level. The architecture is implemented as an open-source coordinator that runs Slither, Mythril, halmos, and ObligationSol against a contract and emits a versioned PARALLAX-5 certificate with full audit trail. The accompanying TOOL-MAPPING v1.0 calibration artifact translates per-tool findings to obligation-graded evidence with justifications.
- **On-chain indistinguishability** (Theorem 19): no on-chain-only monitor can distinguish

worlds with identical observations but differing trust-base assumptions. This justifies the basis-unobservable category as an information-theoretic boundary.

- **Strengthened economic theorem with monitor false negatives** (Proposition 24): $p^* = (1 - c/v)/(1 - \epsilon)$; deterrence by adoption alone becomes mathematically impossible when $\epsilon > c/v$.
- **Two-axis empirical catalog** (Section 10, Appendix supplement `catalog.csv`): 53 incidents totaling \$5.97 billion classified by root cause and basis observability, yielding $L_{\text{basis-observable}} = \4.01B and $L_{\text{basis-unobservable}} = \1.62B .
- **Loss-prevention ceiling with false negatives** (Proposition 25): the gate prevents at most $p(1 - \epsilon)L_{\text{basis-observable}}$; the residual is $L_{\text{basis-unobservable}} + (1 - p(1 - \epsilon))L_{\text{basis-observable}}$.
- **Industry standard** (Section 17): PARALLAX-5 with five compliance levels (P0–P5) and a machine-checkable certificate format.

2 Threat Model and Trust Base

2.1 Trust Base Assumptions

Three off-chain assumptions sit outside the basis:

- OA₁ Key Integrity: signing keys are not exfiltrated, duplicated, or lost.
- OA₂ Signer Sovereignty: multisig and governance signers are not coerced, deceived, or coordinated.
- OA₃ Infrastructure Integrity: RPC nodes, oracle data sources, validator software are not subverted.

Let $\text{TB}(w) := \text{OA}_1(w) \wedge \text{OA}_2(w) \wedge \text{OA}_3(w)$.

2.2 Transition-Level Model

A transition is $t = (s, o, s')$. For each obligation A_i , we define a transition predicate $A_i(t)$. The basis predicate is $B(t) := A_1(t) \wedge A_2(t) \wedge A_3(t) \wedge A_4(t) \wedge A_5(t)$. The violation signature is $\sigma(t) := \{i : \neg A_i(t)\}$.

2.3 Loss Predicate

$\text{Loss}(t, w)$ denotes that transition t in world w causes unauthorized loss, unbacked minting, illicit value extraction, collateral misvaluation, or violation of a protocol-defined protected-value relation.

3 The Five Obligations

3.1 A₁ Value Conservation

Obligation 1 (Value Conservation, A_1). *Every value-affecting transition preserves a protocol-specific conservation relation $C(s, o, s')$ over assets, liabilities, shares, claims, collateral, minted supply, burned supply, fees, and rounding residuals.*

Specializations:

- Vault: $C_{\text{vault}}(s, o, s') := \text{backing}(s') \geq \text{claims}(s') \wedge \text{minLiq}(s') \wedge \text{noZeroShareMint}(s, o, s') \wedge \text{roundingLoss}(s, o, s') \leq \epsilon$.
- Lending: $C_{\text{lending}}(s, o, s') := \text{collateralValue}(s') \geq \lambda \cdot \text{debt}(s')$ (where collateral value depends on A_5).

- Stablecoin: $C_{\text{stable}}(s, o, s') := \text{reserves}(s') + \text{authorizedCredit}(s') \geq \text{supply}(s')$.

The bidirectional formulation matters. An initial one-directional formulation was refuted by Z3, which constructed a counter-witness exploiting integer division-by-zero. The strengthened biconditional removes the pathology.

3.2 A_2 Authorization Closure

Obligation 2 (Authorization Closure, A_2). *Every state-mutating operation requires the caller to satisfy the operation's authorization predicate.*

3.3 A_3 Signature Integrity

Obligation 3 (Signature Integrity, A_3). *Every signature-bound authorization has $\text{ecrecover}(h, v, r, s_\sigma) \neq 0 \wedge \text{ecrecover}(h, v, r, s_\sigma) = \text{expected}$, with h incorporating domain separator, nonce, deadline, chain ID.*

3.4 A_4 Temporal Distinctness

We formalize A_4 with an explicit sequencing dependency relation.

Definition 1 (Sequencing dependency relation). *Let $\Delta \subseteq \mathcal{O} \times \mathcal{O}$ be a protocol-specific relation where $(o_1, o_2) \in \Delta$ iff o_1 must happen-before o_2 , or o_2 must not observe stale or intermediate state from o_1 .*

Obligation 4 (Temporal Distinctness, A_4). *For a transition $t = (s, o, s')$:*

$$\begin{aligned} A_4(t) := & \text{depth}(t) = 0 \wedge \text{noIntermediateRead}(t) \\ & \wedge \text{respectsHappensBefore}(t, \Delta) \\ & \wedge \text{noSameBlockDependencyInversion}(t, \Delta). \end{aligned}$$

This brings MEV/order-dependence cases into formal scope.

3.5 A_5 External-Attestation Trust Boundary

We parameterize A_5 by an explicit adversary model.

Definition 2 (Adversary parameter). *The adversary parameter*

$$\text{Adv} := \langle \text{budget}, \text{latency}, \text{corruptionThreshold}, \text{liquidityAccess}, \text{validatorControl} \rangle$$

parameterizes the adversary against which manipulation-resistance is asserted.

Obligation 5 (External-Attestation Trust Boundary, A_5). *Every state transition consuming an external fact x verifies:*

$$A_5(t) := \text{fresh}(x, \tau_{\max}) \wedge \text{domainBound}(x) \wedge \text{ManipResistant}(x, \text{Adv}) \wedge \text{Quorum}(x, q_{\min}) \wedge \text{Diversity}(x).$$

Specializations:

- Price feed: $\text{ManipResistant}_{\text{price}}$ requires liquidity depth, TWAP window, multi-source aggregation, deviation bound.
- Bridge: $\text{ManipResistant}_{\text{bridge}}$ requires q -of- n independent verifiers, source-chain finality, independent RPC paths.

This generalization brings Kelp DAO's 1-of-1 verifier configuration into the basis as an A_5 violation at configuration time.

4 Conditional Completeness and Falsifiability

Definition 3 (Security-interface adequacy). *The five-obligation interface is adequate for a class of value-bearing systems if*

$$\forall t, w. \text{TB}(w) \wedge B(t) \implies \neg \text{Loss}(t, w).$$

Theorem 4 (Conditional completeness). *Assume adequacy. Then $\forall t, w. \text{TB}(w) \wedge \text{Loss}(t, w) \implies \sigma(t) \neq \emptyset$.*

Theorem 5 (Falsification criterion). *The basis is complete for transition class $\mathcal{C}$ iff no $t \in \mathcal{C}$ admits a basis counterexample (t, w) with $\text{TB}(w) \wedge \text{Loss}(t, w) \wedge B(t)$.*

5 Minimality, Independence, and Closure

Theorem 6 (Transition-level basis minimality). *For every $i \in \{1, \dots, 5\}$, there exists a transition t_i such that $\neg A_i(t_i) \wedge \bigwedge_{j \neq i} A_j(t_i)$.*

Theorem 7 (Constructive closure inhabitation). *Given independence witnesses, every nonempty $S \subseteq \mathcal{A}$ is inhabited by a product transition.*

Proposition 8 (Concrete toy-machine inhabitation). *For each nonempty $S \subseteq \{A_1, \dots, A_5\}$, there exists a single-contract toy value-bearing machine (not a product of independent components) whose transition violates exactly S . This is formalized as the `ToyMachineState` record in our Lean module, with theorem `toy_machine_inhabits_subset`.*

The constructive theorem proves logical independence and absence of formal implication; the toy-machine result shows that inhabitation is not an artifact of free products.

6 Step-Secure Gates

An earlier formulation defined the gate using a post-state-only check, which was incorrect: it checked only the post-state. Post-state-only checks miss A_2 , A_3 , and A_4 violations because these are transition properties — an unauthorized call can leave a perfectly Secure post-state.

6.1 Step Security

Definition 9 (Step security). *For a candidate transition $t = (s, a, s')$:*

$$\text{StepSecure}(t) := \text{StateSecure}(s') \wedge B(t).$$

6.2 The Maximal Safe Gate

Definition 10 (Step-secure gate).

$$G^*(s, a) = \begin{cases} \tau(s, a) & \text{if } \text{StepSecure}(s, a, \tau(s, a)), \\ s & \text{otherwise.} \end{cases}$$

Theorem 11 (Gate state safety). $\text{StateSecure}(s) \implies \text{StateSecure}(G^*(s, a))$.

Theorem 12 (Gate transition safety). *If $G^*(s, a) = \tau(s, a) \neq s$, then $B(s, a, \tau(s, a))$.*

This is the strictly stronger property only the step-secure gate gives. The Lean theorems are `basis_gate_state_safety` and `basis_gate_transition_safety`.

6.3 Maximal Permissive Safety

Theorem 13 (Maximal permissive safety, corrected). *Among all gates H that either execute $\tau(s, a)$ or return s , and that never execute a non-step-secure transition from a secure pre-state, G^* accepts the largest possible set of actions.*

Proof. If G^* rejects a at s (returns s), then $\text{StepSecure}(s, a, \tau(s, a))$ fails. If H executes a ($H(s, a) = \tau(s, a)$), then by the safety hypothesis on H , StepSecure must hold — contradiction. Hence H must also return s . $\square$

The Lean theorem is `basis_gate_is_maximal_permissive`.

6.4 Monitor Soundness Suffices

Theorem 14 (Monitor soundness suffices for gate safety). *If each monitor m_i is sound ($m_i(t) = \text{pass} \implies A_i(t)$), the monitor-implemented gate preserves the basis. No completeness or liveness assumption is required for safety.*

7 Adaptive AI-Agent Safety

Theorem 15 (Adaptive iteration preserves security). *For any agent policy π (adaptive, stochastic, history-dependent, adversarial, self-modifying), iterating G^* under π keeps every reachable state secure.*

Corollary 16 (Agent containment). *For any adaptive agent policy, the reachable state set under G^* is a subset of $\{s : \text{Secure}(s)\}$.*

The theorem does not prove globally safe AI. It proves that, for value-bearing state transitions mediated by sound monitors, the agent cannot drive the system outside the supplied invariant envelope.

8 Cross-VM Refinement

Definition 17 (Step-secure refinement map). *A step-secure refinement map from domain D to abstract V comprises $\alpha : S_D \rightarrow S_V$, $\alpha_A : \mathcal{A}_D \rightarrow \mathcal{A}_V$, and three preservation conditions:*

1. *State predicate preservation:* $\text{StateSecure}_D(s) \iff \text{StateSecure}_V(\alpha(s))$.
2. *Transition simulation:* $\alpha(\tau_D(s, a)) = \tau_V(\alpha(s), \alpha_A(a))$.
3. *Step-security preservation:*

$$\text{StepSecure}_D(s, a, \tau_D(s, a)) \iff \text{StepSecure}_V(\alpha(s), \alpha_A(a), \tau_V(\alpha(s), \alpha_A(a))).$$

Theorem 18 (Gate refinement transfer). *Given a step-secure refinement map, $\alpha(G_D^*(s, a)) = G_V^*(\alpha(s), \alpha_A(a))$, and consequently abstract gate safety transfers to the domain machine.*

Three instantiations: Solana SBPF (lamports/authority keys/CPI depth/sysvars), Move/-Sui (resources/capabilities/txDepth/epochTime — Move’s linear type system enforces no-forgery and no-duplication, but *not* economic solvency or correct business logic), banking ledger (customer balances/authorized clerks/transaction depth/market-rate timestamps).

9 Off-Chain Trust Base and Indistinguishability

Theorem 19 (On-chain indistinguishability of trust-base compromise). *If two worlds w_0, w_1 produce identical on-chain observations $\text{obs}(w_0) = \text{obs}(w_1)$ but differ in trust-base assumptions, then any on-chain-only monitor $M(w) = f(\text{obs}(w))$ gives the same verdict on both: $M(w_0) = M(w_1)$.*

Root cause class	Incidents	Aggregate loss (\$M)	Share
on-chain code/design	38	3,378	56.6%
off-chain signer	5	1,148	19.2%
off-chain key	7	829	13.9%
off-chain infrastructure	3	612	10.3%
Total	53	5,967	100.0%

Table 1: Root-cause summary. Where the failure originated.

Basis observability	Incidents	Aggregate loss (\$M)	Share
basis-observable (gate would reject)	43	4,008	67.2%
ambiguous (monitor-design-dependent)	2	335	5.6%
basis-unobservable (residual)	8	1,624	27.2%
Total	53	5,967	100.0%

Table 2: Basis-observability summary. Whether a sound gate would reject the malicious transition. Basis-observability is not aligned with root cause: 5 entries (\$629M) have off-chain root cause but produce basis-observable on-chain consequences (Resolv, Kelp DAO, PAID Network, EasyFi, bZx Polygon admin) — these are reclaimed by a sound gate even though their root cause is off-chain. Drift is classified separately as ambiguous; see §11.4 for the parameterized treatment.

Corollary 20. *Any system claiming to prevent both on-chain basis violations and off-chain key compromise must use information beyond the on-chain transition trace.*

This makes the irreducible residual $L_{\text{basis-unobservable}}$ precise: it is exactly the loss mass from transitions whose on-chain trace is observationally identical to a legitimate one.

10 Empirical Catalog

We classify 53 historical incidents (2016–2026) along two distinct axes. The full per-incident table with source URLs is in the supplementary `catalog.csv`; here we report aggregate statistics.

10.1 Key Reclassifications

Resolv Labs (2026, \$23M). Root cause: AWS KMS key compromise (off-chain). Consequence: unbacked USR mint, an A_1 violation under *relational* conservation (reserves $\geq$ supply). A sound A_1 gate would have rejected the mint despite the off-chain root.

Drift Protocol (2026, \$285M). Root cause: DPRK social engineering of 2-of-5 multisig (off-chain signer). Consequence chain: (1) fake collateral whitelist, (2) deposit, (3) borrow against fake collateral. Under generalized A_5 with quorum/diversity requirements for collateral onboarding, step (1) is an A_5 violation. Pure on-chain authority check (naive A_2) would not catch it; intent-aware or quorum gates would. Classified as *ambiguous*.

Kelp DAO / LayerZero (2026, \$292M). Root cause: 1-of-1 DVN verifier configuration (off-chain infrastructure). Consequence: bridge released reserves on invalid attestation. Under generalized A_5 requiring q -of- n independent verifiers, the 1-of-1 configuration is an A_5 violation at deploy time, and the malicious bridge release is detectable. Classified as *basis-observable*.

Property	halmos verdict	Witness / paths
A_1 vulnerable (Cream-clone)	FAIL	atkDep=1, donation=524288, victimDep=524288
A_1 hardened (MIN_LIQUIDITY)	PASS over 6 paths	—
A_3 vulnerable (Wormhole-clone)	FAIL	recovered=0, $v = 0$ accepted
A_3 hardened (zero+signer check)	PASS over 4 paths	—
A_5 vulnerable (Mango-clone)	FAIL	price=0, threshold= 2^{255}
A_5 hardened (freshness gate)	PASS over 5 paths	—
A_4 cross-function (Solv-clone)	FAIL	amount=0x8000000000000000
A_4 hardened (sibling guard)	PASS over 3 paths	—

Table 3: halmos symbolic-execution verdicts. Each FAIL is a concrete counter-example trace at the EVM bytecode level.

10.2 Bytecode-Level Reproduction

The basis predicates are not merely abstract: each archetype in the empirical catalog admits a bytecode-level reproduction. We provide four matched pairs of vulnerable and hardened Solidity contracts, each compiled to EVM bytecode and analyzed by halmos symbolic execution at the bytecode level. The vulnerable versions reproduce the relevant archetype; the hardened versions apply the canonical mitigation; halmos verifies the absence of the basis violation in the hardened versions over a bounded symbolic-path budget. The results below are reproducible from `parallax/formal/halmos/verify.sh`; each FAIL verdict is a concrete EVM bytecode counter-example trace, and each PASS is a bounded symbolic-path proof of absence.

10.3 Exploit Normal Form

Definition 21 (Basis normal form). *An exploit report is in basis normal form when it identifies (i) the minimal transition trace producing protected-value loss, (ii) the violated obligation signature $\sigma(t)$, (iii) the root cause class, (iv) the basis observability, (v) the weakest precondition or patch restoring every violated obligation, and (vi) a machine-checkable artifact witnessing the violation or preservation.*

11 Economic Security

11.1 The Per-Attack Theorem

An earlier formulation of this theorem incorrectly stated “under A_2 preservation, attacker profit is zero”; this was inadequate because the attacker could exploit any of A_1, A_3, A_4, A_5 . The corrected statement is:

Theorem 22 (No basis-violating exploit succeeds under a sound gate). *Assume every attack in a class $\mathcal{K}$ succeeds only by executing a transition t with $\neg B(t)$. Under a sound step-secure gate, no attack in $\mathcal{K}$ executes successfully.*

The Lean theorem is `no_basis_violating_exploit` (full name: `no_basis_violating_exploit_under_sound_gate`). The corresponding Z3 result `prove_no_basis_violating_attack_succeeds` returns UNSAT on the joint constraint (gate executes $\wedge$ executed transition violates B).

Corollary 23 (Economic consequence). *For attacks requiring basis violation:*

$$\mathbb{E}[\text{utility}_{\text{verified}}] \leq -c$$

where c is attack cost. “Profit = 0” is only correct when “profit” is defined gross-of-cost.

11.2 Critical Adoption Rate with Monitor False Negatives

Let p be substrate adoption rate, ϵ the conditional false-negative rate (probability a monitor misses an attack given the gate runs), v attack value, c attack cost. Attack succeeds with probability $1 - p(1 - \epsilon)$, giving expected utility $U = (1 - p(1 - \epsilon))v - c$.

Proposition 24 (Critical verification rate). *The rational attacker is deterred iff*

$$p \geq p^* = \frac{1 - c/v}{1 - \epsilon}.$$

Deterrence by adoption alone becomes mathematically impossible when $\epsilon > c/v$; at $\epsilon = c/v$, universal adoption ($p = 1$) is exactly the break-even boundary.

For $v = \$10^7$, $c = \$5 \times 10^4$ (so $c/v = 0.005$):

- $\epsilon = 0$: $p^* = 0.995$ (**99.5%** verified suffices).
- $\epsilon = 0.001$: $p^* \approx 0.996$ (still feasible).
- $\epsilon = 0.005 = c/v$: $p^* = 1.000$ (universal-adoption boundary).
- $\epsilon = 0.01$: $p^* \approx 1.005$ — **impossible**.

The non-obvious finding: even with universal adoption, a monitor false-negative rate strictly above c/v breaks deterrence. Complementary controls (legal penalties, staking forfeiture, chain-level slashing) are then necessary to increase effective attack cost.

11.3 Loss-Prevention Ceiling with the Two-Axis Model

An earlier formulation of this proposition treated L_{off} as the irreducible residual. Under the two-axis model, the correct residual is $L_{\text{basis-unobservable}}$:

Proposition 25 (Loss-prevention ceiling, corrected). *Let L_{basis} be losses whose exploit transition would be rejected by a sound basis gate (regardless of whether the root cause is on-chain or off-chain). Let L_{unobs} be losses whose exploit transition is observationally indistinguishable from a legitimate transition (satisfies the basis; failure entirely in the off-chain trust base). Then a gate with adoption rate p and false-negative rate ϵ prevents at most*

$$p(1 - \epsilon)L_{\text{basis}}$$

of historical losses. The residual is

$$L_{\text{unobs}} + (1 - p(1 - \epsilon))L_{\text{basis}}.$$

11.4 Parameterized Basis-Observability

The binary basis-observability predicate of earlier formulations conflates several distinct information requirements. We parameterize basis-observability by an explicit observation set Ω .

Definition 26 (Observation set). *An observation set $\Omega \subseteq \mathcal{O}$ is the set of facts a monitor can read about a candidate transition t . We define four canonical sets with $\Omega_{\text{chain}} \subset \Omega_{\text{config}} \subset \Omega_{\text{intent}} \subset \Omega_{\text{infra}}$:*

- Ω_{chain} : on-chain state and transaction data only.
- Ω_{config} : Ω_{chain} plus declared protocol configuration (verifier quorum, oracle source set, risk parameters, admin-role membership).
- Ω_{intent} : Ω_{config} plus signer intent, governance proposal metadata, wallet UI context, off-chain co-signed pre-images.

p	Prevented (\$B, \epsilon = 0)	Residual (\$B)	% prevented
0%	0.00	5.97	0.0%
25%	1.00	4.97	16.8%
50%	2.00	3.97	33.6%
75%	3.01	2.96	50.4%
90%	3.61	2.36	60.5%
99%	3.97	2.00	66.5%
100%	4.01	1.96	67.2%

Table 4: Counterfactual loss prevention with $L_{\text{basis}} = \$4.01\text{B}$, $L_{\text{unobs+amb}} = \$1.96\text{B}$. The asymptotic ceiling is 67.2% under the conservative assumption that ambiguous incidents (\$0.34B) are not prevented. A nuanced model treating half of ambiguous loss as preventable (sensitivity analysis only) yields a ceiling of 70.0%. The headline 67.2% figure attributes off-chain-rooted-but-on-chain-consequence losses (\$629M) as preventable; the prior naive on/off-chain split would have classified these as unrecoverable.

- Ω_{infra} : Ω_{intent} plus external infrastructure state (KMS integrity, RPC compromise, DNS status, validator-node health).

Definition 27 (Ω -basis-observability). A transition t is Ω -basis-observable iff there exists a sound monitor M using only observations in Ω such that $M(t) = \text{reject}$ on every basis-violating transition matching t ’s archetype.

Under this refinement, several classifications that were previously “ambiguous” become principled. Drift Protocol’s fake-collateral-whitelist incident is Ω_{chain} -unobservable (the on-chain action was an authorized admin call) but Ω_{config} -observable (the declared collateral-onboarding policy requires a multi-stage review and time delay that this transition skipped). Resolv Labs’ unbacked mint is Ω_{config} -unobservable (the protocol configuration permits the mint authority) but Ω_{intent} -observable (an intent-aware monitor with declared session-policy on minting would have rejected). CoW Swap’s DNS hijack is genuinely Ω_{infra} -unobservable: the victim signs legitimately, and no monitor without breaking trust-base assumptions about DNS itself can distinguish.

Minimum observation set	Entries	Loss (\$M)
Ω_{chain}	41	3,599.5
Ω_{config}	5	1,367.0
Ω_{intent}	0	0.0
Ω_{infra} -unobservable (genuinely irreducible)	7	999.5

Table 5: 53-incident catalog re-classified by minimum required observation set. The bulk of historical loss is recoverable at Ω_{chain} alone; Ω_{config} extends reach to bridge configuration and oracle policy. Genuinely irreducible loss is bounded at \$1.0B (16.7% of total). Reproducer at `parallax/formal/observation_sets.py`.

This parameterization sharpens the framework’s claims: rather than a binary “basis-observable / not”, each incident has a *minimum information requirement* for the monitor that would have caught it. Monitor designers select Ω appropriately; protocols select an appropriate compliance level given their available monitoring infrastructure.

11.5 Unified loss ledger

The catalog supports two complementary aggregations: a coarse *binary* basis-observability summary (Table 2, retained for backward compatibility) and a refined *observation-set* decomposition (Table 5, the headline model). To prevent confusion across these views, Table 6 consolidates the headline quantities with one explicit definition per row. The observation-set model is treated as primary; the binary view is a derived projection.

Quantity	Loss (\$B)	Meaning
$L_{\Omega_{\text{chain}}}$	3.60	rejectable with chain-only monitor (41 incidents)
$L_{\Omega_{\text{config}} \setminus \Omega_{\text{chain}}}$	1.37	additionally rejectable with declared-configuration monitor (5 incidents)
$L_{\Omega_{\text{intent}} \setminus \Omega_{\text{config}}}$	0.00	additionally rejectable with intent-aware monitor (0 incidents in current corpus)
$L_{\text{irreducible}}$	1.00	not rejectable without infra/trust-base information (7 incidents)
$L_{\text{aggregate}}$	5.97	total aggregate loss (53 incidents)

Table 6: Unified loss ledger. The observation-set decomposition (rows 1–4) is exhaustive and disjoint; row sums to the aggregate. The earlier binary model’s \$0.34B “ambiguous” bucket (Table 2) corresponds to monitor-design-dependent edge cases now distributed across Ω_{config} depending on which configuration facts the monitor reads; we treat ambiguous as a sensitivity overlay rather than a headline category.

The headline loss-prevention story rephrased against the observation-set model: with chain-only monitoring a sound gate prevents up to \$3.60B; extending to declared-configuration monitoring brings the preventable loss to \$4.97B; the residual \$1.00B requires information outside any of the monitor classes considered in this paper. The earlier $L_{\text{basis}} = \$4.01\text{B}$, $L_{\text{unobs+amb}} = \$1.96\text{B}$ split of Proposition 25 (Table 4) is the same data under coarser binning, with \$0.96B of what the binary model called “unobservable + ambiguous” becoming reachable at Ω_{config} under the refined model. Table 6 is the canonical reference.

12 The Walkaway Property

The economic-security analysis of Section 11 bounds adversary cost-to-attack but leaves open a structural question: *what does it mean for a DeFi protocol to operate without continued action by its founders?* Industry vocabulary uses the terms “decentralized,” “trustless,” and “immutable” interchangeably, but these terms are not formal predicates over protocol artifacts. In this section we define the *walkaway property* as a falsifiable predicate, present a five-level classification, and prove four supporting theorems.

12.1 Definition and motivation

Industry vocabulary mixes two distinct concerns. The first is whether the protocol’s bytecode is on chain (which all deployed contracts are). The second is whether the protocol’s continued operation requires action by any specific party. We formalize the second.

Definition 28 (Walkaway property). *A deployed DeFi protocol P satisfies the full walkaway property iff every obligation $A_i \in \{A1, \dots, A5\}$ in the protocol’s certified obligation set is preserved under the operational scenario in which every off-chain party associated with P ceases to act. Symbolically, writing $\text{Adm}(P)$ for the set of off-chain parties holding any privileged role in P ,*

$$\text{Walkaway}_{\text{full}}(P) \equiv \forall A_i \in \text{Obl}(P). \forall \sigma. \text{Adm}(P)\text{-quiescent}(\sigma) \implies \models_{\sigma} A_i.$$

The definition is structural: it does not invoke “decentralization” as a primitive. A protocol with an admin key that can pause withdrawals does not satisfy $\text{Walkaway}_{\text{full}}$, because in the

admin-quiescent scenario, withdrawals remain paused and A1 (value conservation, for users) fails on the relevant transition class. A protocol with an upgrade proxy does not satisfy $\text{Walkaway}_{\text{full}}$ for the same reason.

12.2 Five-level classification

A binary walkaway predicate would be too coarse for real protocols. We define five classes.

Definition 29 (Walkaway class spectrum). *A protocol P admits exactly one of the following walkaway classifications:*

- **Full** — $\text{Walkaway}_{\text{full}}(P)$ holds. No off-chain party retains an irreplaceable role in any obligation’s continued satisfaction.
- **Bounded** — $\text{Walkaway}_{\text{full}}(P)$ fails only for liveness-class obligations; safety-class obligations all hold under admin quiescence. The protocol’s continued operation depends on off-chain infrastructure (e.g., validator set, oracle relayer), but no off-chain party can extract value from quiescent users.
- **Partial** — some safety-class obligations are admin-preserved but the admin set has a structural exit (e.g., timelock with bounded delay).
- **Centralized** — an off-chain party retains the structural ability to violate a safety-class obligation indefinitely.
- **Fake** — the protocol presents the appearance of walkaway (e.g., “immutable contract,” “ownership renounced”) but a residual off-chain role exists that admits safety-class obligation violation (e.g., a proxy whose admin slot was nominally renounced but is reachable via an alternative pathway).

The last class is significant. Inspection by structural pattern alone (does the contract have an `owner()` slot? does it have a proxy?) is insufficient for high-stakes use, because the absence of overt admin patterns does not preclude latent off-chain control. The classification must be assigned by inspection of the obligation-satisfaction trace under quiescence, not by surface structural heuristic.

12.3 Mechanized theorems

We mechanize five supporting results in Lean 4 (see file `lean/Parallax5/Walkaway.lean`; 340 lines, 5 theorems, 3 examples, 11 definitions, 3 structures, 2 inductives; zero `sorry`).

Theorem 30 (Empty-admin walkaway is trivial). *If $\text{Adm}(P) = \emptyset$, then $\text{Walkaway}_{\text{full}}(P)$ holds.*

Theorem 31 (Admin removal preserves obligations). *For any obligation A_i whose satisfaction does not reference $\text{Adm}(P)$, removing parties from $\text{Adm}(P)$ preserves $\models A_i$.*

Theorem 32 (Bounded walkaway eventual stability). *Under a bounded-delay timelock with delay τ , after time $t > \tau$ following the last admin action, the protocol’s safety-class obligations hold under admin quiescence.*

Theorem 33 (Centralized exclusion). *If $\exists A_i \in \text{Obl}(P)$ such that A_i admits a violation under any single-party action by an element of $\text{Adm}(P)$, then $\text{Walkaway}_{\text{full}}(P)$ does not hold.*

Theorem 34 (Minimal admin-free CPMM is full walkaway). *A constant-product AMM with no factory, no protocol-fee switch, and no admin role has $\text{Adm}(P) = \emptyset$ on its full obligation set and therefore satisfies $\text{Walkaway}_{\text{full}}$. This is the substrate’s cleanest positive example: full walkaway holds because no privileged role exists to remove.*

Theorem 35 (Uniswap V3 Core pool admits full walkaway under scoped obligations). *A deployed Uniswap V3 Core pool, scoped to its swap/mint/burn entry points and excluding the factory-level protocol-fee switch, has $\text{Adm}(P) = \emptyset$ on the scoped obligation set $\{A1, A2, A4\}$ and therefore satisfies $\text{Walkaway}_{\text{full}}$ under that scope.*

Theorems 34 and 35 are worked positive examples, verified by mechanical check that the relevant deployed bytecode has no admin slot reachable from any function selector with state-mutating effect on the obligations in scope. The two theorems differ in their scoping discipline: the minimal CPMM is admin-free by construction (no factory, no protocol fee), so its walkaway claim is unrestricted. Uniswap V3 Core, by contrast, has a factory-level protocol-fee switch controlled by governance; the scoped theorem here applies to *pool-level safety only*, declaring factory-level commercial parameters out of scope. Protocols that wish to publish a Uniswap-V3-style walkaway certificate must declare their scope explicitly in the certificate’s `walkaway.scope` field. Hostile reviewers should challenge the scoping, not the theorem; the scoping is the substantive claim, and the substrate prefers narrow honest claims over broad informal ones.

12.4 Relation to existing terminology

The walkaway property is strictly weaker than “fully autonomous” (which would require liveness-class obligations also to hold under quiescence, which is impossible for any protocol depending on external information or coordination) and strictly stronger than “no admin functions in the visible ABI” (which does not preclude proxy-mediated admin access). It is the right level of structure to make claims about founder-independent operation falsifiable. A protocol’s walkaway classification is one of the certificate fields specified in Section 22.

13 2026 Forward-Test

The empirical catalog (53 incidents, 2016–2026) used to define the framework risks overfitting: it is possible that classification choices were made in hindsight. To test predictive validity, we applied PARALLAX-5 to 13 additional 2026 incidents that were not in the catalog at the time of v3 framework definition, totaling \$725.7M in losses. The obligation schema and basis predicates were frozen prior to classification of these incidents; classification was performed without modifying the basis. We emphasize that the attack vectors were public at classification time, so this is a schema-stability test rather than a prediction-of-attack test. A refutation requires a trust-base-respecting, loss-inducing transition that satisfies all five obligations. None of the 13 incidents constitutes a refutation.

	Confirmed	Partial	Refuted	Pending
Count	13	0	0	0
Loss \$M	725.7	0.0	0.0	0.0

Table 7: Forward-test of 13 2026 incidents. Zero refutations: no incident was a basis-respecting, trust-base-respecting loss. Note that this is a *schema-stability test*, not a prediction-of-attack test: the attack vectors were public at classification time, but the obligation schema was frozen before classification. The result establishes that the schema remained adequate without modification under thirteen subsequent incidents.

The CoW Swap DNS hijack (\$1.2M, April 2026) correctly classifies as basis-unobservable: victims signed legitimate transactions on a hijacked frontend, and the basis cannot distinguish their intent. This validates the `L_basis-unobservable` category as bounded and well-defined — the boundary cuts where we said it would. All other 2026 incidents are basis-observable,

including the two largest (Kelp DAO \$292M and Drift Protocol \$285M), confirming that the framework’s reach extends to fresh, post-hoc-classification-free incidents.

The 2026 obligation frequency (A_2 : 7, A_5 : 6, A_1 : 4, A_4 : 0, A_3 : 0) matches the 2024–2025 distribution: authorization (key compromise) and external-attestation (bridge / oracle) dominate; pure reentrancy and pure signature forgery are increasingly rare. The full data is at `parallax/formal/forward_test/forward_2026.py`.

14 PARALLAX-5 as a Unification Layer

The smart-contract security tooling ecosystem is fragmented. Slither [7] produces hundreds of detector verdicts in syntactic format; Mythril [8] and halmos [9] produce symbolic counter-examples; Certora [10] produces rule-level UNSAT/SAT verdicts; Lean and Coq produce theorem-level proof artifacts; auditor reports use a narrative format. There is no common interface. A protocol team adopting all five gets five incompatible reports with no shared vocabulary.

PARALLAX-5 is designed as a *coordination layer* for these tools, not a competitor to any of them. The five obligations $A_1, \dots, A_5$ provide a fixed common vocabulary; every existing tool maps into specific obligations and specific compliance levels.

Tool	Role in PARALLAX-5	Compliance level produced
Slither	Static detectors emit obligation hints	P2 (Statically Screened)
Mythril, Manticore	Bounded symbolic exploration	P3 (Symbolically Checked)
halmos	Foundry-integrated EVM bytecode symbolic execution	P3
Certora	Rule-level SMT proofs over CVL specifications	P4 (Formally Proved)
Coq, Lean	Theorem-level meta-proofs	P4
KEVM [11]	K-framework operational semantics	P4 over the EVM
PARALLAX runtime gate	Step-secure execution-time enforcement	P5 (Runtime Enforced)
Auditor narrative	Maps each finding to $\sigma(t)$	augments any level

Table 8: PARALLAX-5 as coordination layer. Each existing tool maps cleanly into one or more obligations and a specific compliance level. No tool is displaced; outputs compose.

The contribution is not better detection but *better organization*. The certificate format (Section 17) provides a single artifact where verdicts from heterogeneous tools compose by obligation; the validator (`parallax/standard/validator.py`) checks that they compose consistently. A protocol that runs Slither + Certora + halmos and a runtime gate produces a single P5 certificate referencing all four tools’ artifact hashes.

This positioning matters for adoption: PARALLAX-5 does not ask any tool vendor or auditor to abandon their stack; it asks them to label outputs by obligation. Tool vendors gain interoperability; auditors gain a structured report format; protocols gain a single artifact; insurers gain a comparable rating; AI-agent platforms gain a uniform execution-time gate. All five constituencies benefit without anyone losing.

15 The PARALLAX-CROPS Trust-Surface Vector

The depth scale $D_0, \dots, D_5$ of Section 3 produces a single security rating per obligation. For a consumer who wants to know “how strong is this protocol’s signature handling,” or “does this AMM have any meaningful censorship-resistance,” a single scalar is insufficient. In this section we define a five-component trust-surface vector and prove its derivation from the obligation-coverage data.

Relation to the Ethereum Foundation’s CROPS. The Ethereum Foundation’s 2026 strategic communications use *CROPS* as a mnemonic for “censorship-resistant, open source, private, secure” — four values stated as ecosystem-level priorities. We use *PARALLAX-CROPS* as the name for our five-component refinement, which expands *capture-resistance* into a separate dimension R distinct from *censorship-resistance* C . The two are related (a captured protocol can censor its users) but not identical, and the substrate’s walkaway property feeds the R dimension uniquely. We label our vector *PARALLAX-CROPS* to make this refinement explicit and to avoid the appearance of citing the EF mnemonic as a five-letter acronym.

15.1 The five dimensions

Definition 36 (CROPS dimensions). *A protocol’s CROPS vector is the tuple $(C, R, O, P, S) \in D^5$ where each component is a depth in the $D_0, \dots, D_5$ scale:*

- C — Censorship-resistance. *The protocol’s resistance to value flow being blocked or redirected by external parties.*
- R — capture-Resistance. *The protocol’s resistance to off-chain parties acquiring or retaining unilateral control over its operation.*
- O — Openness. *The accessibility of the protocol’s source, specification, and operational artifacts to external review.*
- P — Privacy. *The protocol’s preservation of user information against external observation (transaction-graph privacy, signature non-malleability, oracle-input privacy).*
- S — Security. *The aggregate strength of the protocol’s safety properties, summarizing across all obligations.*

These five dimensions are not chosen arbitrarily: they correspond exactly to the technical priorities articulated for Ethereum’s longevity-over-breadth phase, in which security and capture-resistance are co-equal with throughput considerations rather than subordinate to them.

15.2 The contribution matrix

Each obligation $A_1, \dots, A_5$ contributes to a specific subset of CROPS dimensions. The matrix is sparse and conservative: an obligation contributes to a dimension only when the obligation’s satisfaction is a necessary condition for that dimension to be meaningful at any depth $\geq D_1$.

Definition 37 (Contribution matrix). *The contribution function $\text{contrib} : \{A_1, \dots, A_5\} \rightarrow \mathcal{P}(\{C, R, O, P, S\})$ is defined by:*

	C	R	O	P	S
A_1 value conservation	✓	–	–	–	✓
A_2 authorization closure	–	✓	–	–	✓
A_3 signature integrity	–	–	–	–	✓
A_4 temporal distinctness	✓	–	–	–	✓
A_5 external-attestation trust	✓	–	✓	–	✓

On the privacy contribution of A_3 . A_3 does not contribute to P in the matrix above. Signature malleability is an integrity, replay, and canonicalization issue; the privacy-adjacent consequence (cross-message identity correlation by an attacker) is real but conditional on the protocol making an explicit privacy claim via signature canonicalization (ring signatures, blinded signatures, selective disclosure). Honest PARALLAX-CROPS reporting requires that privacy contributions flow from explicit privacy primitives declared in the specification, not from incidental properties of A_3 evidence. Protocols whose A_3 enforcement is part of a privacy-primitive design should declare `privacy_primitives_depth` accordingly.

Three derived dimensions extend the matrix beyond per-obligation contributions:

- The **walkaway classification** contributes to R via $\text{depthOfWalkaway}(\text{class}) \in \{0, 1, 3, 4, 5\}$ for $\{\text{fake}, \text{centralized}, \text{partial}, \text{bounded}, \text{full}\}$ respectively.
- The **source-openness depth** (whether the contract source, build inputs, and deployment metadata are publicly available and reproducible) contributes directly to O .
- **Privacy-primitives depth** (whether the protocol uses signature blinding, mixers, ZK commitments, or other privacy-enhancing primitives at the obligation-satisfaction layer) contributes *as the sole source of* P .

15.3 Projection function

Definition 38 (CROPS projection). *For a protocol P with obligation depth assignment $d : \{A_1, \dots, A_5\} \rightarrow D$, walkaway classification w , source-openness depth o , and privacy-primitives depth π , the CROPS vector is computed by maximum over contributors:*

$$\text{CROPS}(P) = \left(\max_{A_i: C \in \text{contrib}(A_i)} d(A_i), \max_{A_i: R \in \text{contrib}(A_i)} \{d(A_i)\} \cup \{\text{depthOfWalkaway}(w)\}, \dots \right).$$

The projection is monotone: increasing any obligation depth or upgrading the walkaway classification can only weakly increase the CROPS vector. This monotonicity is the multi-dimensional analogue of Theorem 48.

15.4 Uniswap V3 Core

Applying Definition 38 to the obligation-depth data of Theorem 35 (which gives $d(A_1) = D_4$ via static and symbolic execution, $d(A_2) = D_5$ via empty-admin verification, $d(A_4) = D_4$ via reentrancy analysis, and walkaway class $\text{full} \Rightarrow R = D_5$, with source openness D_5 because the repository, build inputs, and deployment metadata are public and reproducible, and privacy primitives D_0 because Uniswap V3 makes no privacy claims), we obtain:

$$\text{CROPS}(\text{Uniswap V3 Core}) = (C=4, R=5, O=5, P=0, S=5).$$

Two features of this result deserve comment. First, $P = 0$ is not a defect but an honest reporting: Uniswap V3 makes no privacy claims, and the certificate reflects this rather than inflating the rating. Second, the achievement of $R = D_5$ depends on the walkaway property being verified rather than asserted, which is the content of Section 12.

15.5 Consumer-side policy semantics

A PARALLAX-CROPS vector enables policy-based filtering at the consumer side. A treasury policy might require $C \geq 3$ and $R \geq 4$ and $S \geq 4$ for any pool the treasury allocates to; a privacy-seeking user might require $P \geq 3$. These policies are uniform predicates over the CROPS vector and can be evaluated mechanically against certificates without bespoke per-protocol diligence. The executable specification is provided in `src/parallax5.coordinator/crops.py`; the 19-test suite (`tests/test_crops.py`) covers contribution-matrix consistency, the v1.0.1 invariant that A_3 does not inflate P , the projection function's monotonicity, and consumer-policy worked examples.

16 Worked Examples

Three worked examples accompany this paper, each fully reproducible from a single command (`make demo-vault`, `make demo-bridge`, `make demo-agent`) and consisting of: deliberately-vulnerable source contracts, a faithful Python state-machine simulator demonstrating the attack, patched source contracts, a Lean 4 proof artifact for the patched version, a PARALLAX-5 certificate emitted by the coordinator, a validated certificate, and a registry-submission payload. All four Lean files involved in this section contain zero `sorry`.

16.1 The ERC-4626 inflation attack

A well-documented exploit on naive ERC-4626 vaults: an attacker deposits a single wei, then directly transfers a large asset balance to the vault address (bypassing the deposit accounting), inflating the share price to the point where subsequent victim deposits round down to zero shares. The victim’s assets become recoverable by the attacker via redemption of their original single share.

The mitigation is the virtual-shares pattern introduced in OpenZeppelin v4.8: `convertToShares` uses $(assets \times (totalSupply + V_{sh})) / (totalAssets + V_{as})$ with $V_{sh} = 10^6$ and $V_{as} = 1$. The virtual offset ensures the rounding always favors the vault.

The Lean 4 file `demos/vault/proof/Conservation.lean` (196 lines, 5 theorems, 0 `sorry`) establishes a formal cost lower bound:

Theorem 39 (Inflation-attack cost lower bound). *For the patched vault V and any nonzero deposit amount $a > 0$, if `convertToShares( $V, a$ ) = 0`, then*

$$V.totalAssets + V_{as} > a \cdot (V.totalShares + V_{sh}).$$

For a victim deposit of 10^{18} wei, an attacker wishing to force zero-share rounding must inflate $V.totalAssets$ to at least 10^{24} wei: economically infeasible for any realistic ERC-20. The Python simulator confirms this mechanically: on the vulnerable version, an attacker captures 10^{18} wei; on the patched version, the victim’s economic loss is bounded at 2.5×10^{-7} (roundoff only).

Certificate. The emitted certificate has CROPS vector ($C=4, R=5, O=5, P=0, S=4$) with walkaway classification *full*. Slither finds seven static issues on the vulnerable version and five on the patched (the two missing are `incorrect-equality` checks removed by the virtual-shares fix), but *neither set identifies the inflation attack itself*: the A1 depth of D_4 on the patched version comes from the Lean proof, not from Slither. This is the substrate’s central claim demonstrated concretely: static analysis is necessary but not sufficient for D4 evidence; the certificate composes the heterogeneous evidence honestly.

16.2 Bridge attestation

Cross-chain bridges have produced the largest single losses in DeFi: Wormhole (\$320M, 2022), Ronin (\$625M, 2022), Nomad (\$190M, 2022), Multichain (\$126M, 2023). The common failure mode is attestation-validation failure: the bridge accepts attestations it should not have accepted. We address two vector classes.

Signature malleability (A3). Pre-EIP-2 ECDSA admits two valid signatures (r, s) and $(r, n - s)$ for the same message and public key. A bridge that uses `ecrecover` without enforcing $s \leq n/2$ accepts both. The mitigation is a single check: `require(uint256(s) <= uint256(SECP256K1N_DIV_2), "high-s rejected")`.

Stale-attestation acceptance (A5). A bridge with no freshness window accepts old attestations indefinitely, even after the validator set has rotated due to a discovered compromise. The mitigation combines (i) a freshness window (e.g., one hour) and (ii) an epoch-binding message hash that incorporates the current validator set version.

The Lean 4 file `demos/bridge/proof/Attestation.lean` (193 lines, 6 theorems, 0 `sorry`) establishes:

Theorem 40 (Patched bridge rejects malleable, stale, and future attestations). *Let `patchedBridgeAccepts( $att, t, \sigma, v$ )` $\equiv$ `isFresh( $att, t$ )` $\wedge$ `isLowS( $\sigma$ )` $\wedge$ v . Then for any signature σ and any attestation att at time t , if either $\neg \text{isLowS}(\sigma)$ or $\neg \text{isFresh}(att, t)$, then $\neg \text{patchedBridgeAccepts}(att, t, \sigma, v)$.*

Together with the epoch-rotation lemma (`epoch_rotation_breaks_replay`), this theorem rules out the malleability, stale, future, and rotated-epoch replay families.

Certificate. The bridge’s PARALLAX-CROPS vector is $(C=4, R=4, O=5, P=0, S=4)$ with walkaway classification *bounded*. Two features deserve comment. First, $R = 4$ rather than 5: a bridge’s purpose is to relay attestations from off-chain validator infrastructure, so the bridge cannot achieve full walkaway by structure; its bounded classification, with `dependencies_disclosed` enumerating “off-chain validator infrastructure must continue to attest withdrawals,” is the honest reporting. Second, $P = 0$ reflects the bridge making no privacy claim. Although the patched bridge enforces low-s ECDSA (which has a privacy-adjacent consequence in the form of cross-message identity correlation by an attacker), the v1.0.1 contribution matrix does not inflate P from A_3 evidence alone: privacy contributions flow only from explicit privacy primitives declared in the spec. A bridge using A_3 as part of a privacy-primitive design (e.g., a ring-signature attestation scheme) would declare `privacy_primitives_depth` and earn a non-zero P .

16.3 An AI-agent runtime gate

The third demonstration addresses a class the industry is just beginning to confront at scale: AI agents with on-chain authority. The architectural setting: an autonomous agent (yield manager, treasury rebalancer) is given a private key authorizing it to interact with a vault. Without further structure, every transaction the agent emits is at the user’s risk, with no mitigation available short of revoking the agent’s authority entirely.

The runtime gate G is a contract sitting between the agent and the target vault. Every agent action a is routed through G , which evaluates a *step-secure predicate* $\sigma(s, a)$ over the current vault state s and the candidate action; only actions satisfying σ are forwarded. The predicate enforced by the demonstration’s gate consists of (i) a per-transaction outflow cap (5% of vault balance), (ii) a 24-hour rolling outflow budget (20% of vault balance), (iii) a whitelist of approval targets, set at construction and immutable, (iv) an approval-amount bound.

The Lean 4 file `demos/agent_gate/proof/Containment.lean` (261 lines, four substantive theorems, 0 `sorry`) establishes:

Theorem 41 (Single-step containment). *For any policy Π , any vault state s , and any agent action a , either $\text{gateStep}(\Pi, s, a) = \text{applyAction}(s, a)$ and $\sigma(\Pi, s, a) = \text{true}$, or $\text{gateStep}(\Pi, s, a) = s$ and $\sigma(\Pi, s, a) = \text{false}$. There is no third possibility.*

Theorem 42 (AI-Agent Containment). *For any policy Π , any state s , and any agent action a , if the gate’s transition $\text{gateStep}(\Pi, s, a)$ produces a state distinct from s (i.e., the action was applied), then: every transfer’s amount is bounded by $\Pi.\text{maxOutflowPercent} \cdot s.\text{balance} / 100$; every approve targets a Π -whitelisted spender; every approve amount is bounded by $\Pi.\text{maxApproval}$.*

The significance of Theorem 42 is that its hypothesis quantifies over a *abstractly*: the theorem makes no assumption about the agent’s origin (a human, a language model, a jailbroken language model, an external party in possession of the agent’s private key, a hallucinated tool call). The gate’s guarantees hold over agent-behavior space, not over agent-intent space. This is the formal expression of “the runtime gate makes the agent’s transaction surface safe even under arbitrary adversarial agent behavior.”

Certificate. The runtime gate’s CROPS vector is $(C=5, R=5, O=5, P=0, S=5)$ with walkaway classification *bounded*. This is the only certificate in the demo set to reach D_5 on any obligation. The D_5 assignment is precise: the certified step-secure predicate is enforced at runtime on every submitted action by the deployed gate, with the proof artifact (Theorem 42) establishing the predicate’s containment property and the deployed relay supplying enforcement. Deployment is therefore not itself the proof—a deployed gate can be buggy, misconfigured, or bypassed—but *deployment is the enforcement mechanism whose correctness the proof artifact certifies*. The Python simulator demonstrates this mechanically across five scenarios (legitimate rebalance permitted; max-uint approval to non-whitelisted contract rejected; 100%

single-transfer drain rejected; repeated 5%-cap drains bounded by daily budget; non-whitelisted approval rejected), each behaving as predicted.

16.4 What the demos jointly show

Each demo’s certificate composes heterogeneous evidence (Slither static findings, Lean theorems, walkaway classification, source-openness declaration) under the single 19-field schema specified in Section 22. The CROPS vectors differ across the three protocols in ways that reflect their actual structural properties, not in ways that reflect marketing framing. The substrate’s vocabulary (A1–A5, D0–D5, the five walkaway classes, the CROPS dimensions) is sufficient to describe protocols across three distinct categories (vault, bridge, agent runtime) without extension.

17 Comparison with Existing Standards

PARALLAX-5 is designed to coordinate with, not displace, existing standards. The full mapping is at `paper/STANDARDS.COMPARISON.md`; in brief:

Standard	Scope	PARALLAX-5 mapping
EEA EthTrust [S]/[M]/[Q]	Solidity	$\approx$ P1 / P2 / P3
OWASP SCSVS V1–V13	Solidity	Each chapter clusters under A1–A5 or OA1–OA3
ToB / OpenZeppelin / Consensus	Multi-VM	Findings tag with A1–A5
NIST SP 800-zzz (draft)	DLT	Aligns with OA1/OA2/OA3 trust-base
Immunefi / Code4rena severity	Per-finding	Severity $\leftrightarrow$ obligation violation strength

Table 9: Mapping of existing standards onto PARALLAX-5. Each is unchanged; PARALLAX-5 provides the per-obligation labeling layer they lack.

17.1 Standard Infrastructure

The standard infrastructure is implemented as operational code rather than documentation:

Falsification challenge as CLI. The basis counterexample challenge (`paper/FALSIFICATION_CHALLENGE.md`) is operationalized by `parallax5 challenge validate`, which structurally verifies submissions: the trust base must hold, all five obligations must be claimed satisfied, and protected-value loss must be demonstrated. Submissions failing these checks are rejected as not-refuting (basis hits, not basis misses) with a documented reason. An append-only `parallax5 challenge status` registry tracks submissions across the campaign.

Revocation registry. Certificate lifecycle events (`parallax/standard/revocation.py`) are recorded in an append-only JSONL log with cryptographic content-addressing of the certificate. The events — `issue`, `revoke`, `supersede`, `revalidate` — are immutable; the registry exposes `is_revoked`, `is_superseded`, and `status` for downstream consumers (insurers, CI workflows). Production deployments would back this with on-chain Merkle accumulators for tamper-evidence.

Audit-to-certificate conversion. `parallax5 audit-import` converts narrative or structured audit outputs into PARALLAX-5 certificates. Three input formats are supported out of the box: structured per-finding JSON (`parallax-audit-v1`), SARIF 2.1.0 (the GitHub-compatible static-analyzer format), and native Slither JSON. The converter classifies findings to obligations via keyword analysis, determines a compliance level from severity and tool diversity, and emits a certificate that validates against the schema. A worked end-to-end round-trip from a Trail-of-Bits-style report to a validated certificate is included as a fire test.

Pre-transaction simulator. `parallax5 pretx-simulate` dry-runs the step-secure gate semantics on a candidate transition (sender, target, calldata, value, call depth, oracle context) and emits per-obligation accept/reject with reasons. This is the kernel of the P5 runtime gate experience: every proposed transaction is adjudicated against the certificate’s basis before signing. The reference implementation is in Python; production deployments would re-implement the same semantics in Solidity, Move, and Solana SBPF, with the Python version serving as the conformance oracle.

Together these four pieces transform the standard from documentation into a usable substrate. Each component is exercised by at least one fire test; integration round-trips are verified as part of the self-test.

18 The EVM Backend

The substrate of §3–§6 is stated against an abstract value-bearing state machine. For its theorems to bind on deployed bytecode, the substrate must compose with a production EVM semantics. This section establishes that composition: we exhibit a typeclass-based refinement of the substrate against Nethermind’s `EVMYulLean`—a Lean 4 executable formal model of the Ethereum Virtual Machine and Yul intermediate language at the Cancun hard fork—and report a mechanically-checked instance lifting nineteen of our abstract refinement theorems to `EvmYul.EVM.State`. The verification artifact, with full cryptographic anchoring and reproducibility evidence, is permanently deposited [1].

18.1 Choice of backend

Two backends were considered. The K-Framework EVM semantics [11], mature and Foundry-native through Kontrol, would require a cross-prover translation layer: K is not Lean. `EVMYulLean` is native Lean 4 and reports 99.99% pass rate on the Ethereum Foundation conformance test suite at the Cancun fork.¹ Because the substrate is itself Lean 4, `EVMYulLean` reduces refinement to typeclass instantiation rather than cross-prover translation; this is the path taken here. A K-Framework backend remains available as a complementary path through in-progress KEVM–Lean equivalence work.

18.2 The `EvmLikeMachine` typeclass

The typeclass `EvmLikeMachine S` (Table 10) captures the minimal operational interface required by the substrate’s refinement layer. Six methods suffice: a step function returning an optional successor state, two value-tracking projections (`totalSupply`, `balanceOf`), a sender projection for A_2 , a depth projection for A_4 , and a freshness oracle adapter for A_5 .

18.3 The refinement theorem and its instantiation

We state the central result of this section, then sketch the proof constructively from the deposited artifact.

Theorem 43 (EVMYulLean refinement). *Let $\mathcal{R}$ denote the abstract refinement layer of §6—the set of nineteen Lean 4 theorems stated parametrically over a type S equipped with $[EvmLikeMachine S]$ and a gate $(g : AbstractGate S)$. There exists an instance `EvmLikeMachine (EvmYul.EVM.State)` under which every theorem of $\mathcal{R}$ lifts to a compiled Lean 4 proof term over `EvmYul.EVM.State`. Five further theorems are exhibited over concrete default-constructed values of this type, demonstrating non-vacuity of the lifted set.*

¹Nethermind, `EVMYulLean`, <https://github.com/NethermindEth/EVMYulLean>.

Method	Obligation served
<code>step : S → Option S</code>	step semantics (all)
<code>totalSupply : S → Nat</code>	A_1 value conservation
<code>balanceOf : S → Address → Nat</code>	A_1 per-account
<code>sender : S → Address</code>	A_2 authorization
<code>callDepth : S → Nat</code>	A_4 temporal distinctness
<code>attestationFresh : S → Address → Nat → Bool</code>	A_5 external attestation

Table 10: The `EvmLikeMachine` interface. Any EVM-shaped semantics exposing these operations may be registered as an instance; every parametric theorem of the abstract refinement layer (§6) then lifts to the concrete state type by typeclass elaboration, with correctness enforced by the Lean 4 kernel.

Proof (constructive, externally verifiable). The instance is provided in the deposited module `Instance.lean`; each of its six methods wraps the corresponding `EVMYulLean` projection (§18.4). The lifted theorems are realized as twenty-four declarations in the deposited `TheoremTransfer.lean`; their compiled proof terms are persisted in `TheoremTransfer.olean`. The Lake build of the integrated module set terminates with exit code zero, which is necessary and sufficient evidence that the Lean 4 kernel has accepted every typeclass elaboration, theorem application, and proof term in the closure. The build is reproducible: three independent executions against identical upstream commits produced byte-identical compiled artifacts for the modules whose sources were unchanged (§18.5). A reader may re-verify the artifact independently by executing the included `verify_receipt.py` script against the deposited `receipt.json`; expected output: `OVERALL: VALID` [1]. $\square$

The nineteen lifted theorems partition into five families. (i) Five single-step gate-safety theorems establish rejection of transitions violating A_2 (unauthorized sender), A_4 (non-zero call depth under enabled reentrancy obligation), A_5 (stale oracle attestation), and A_1 (supply-inflating transitions), together with the soundness-trivial case of a fully-disabled gate. (ii) Seven multi-step trace-safety theorems establish that the recursion structure of the trace predicate is well-formed (zero-step base case, head extraction, tail extension, full-trace acceptance under a permissive gate, and full-trace rejection under both reentrancy and unauthorized-sender violations). (iii) Two monotonicity-and-refinement theorems establish that disabling an obligation produces a strictly more permissive gate, and that gate decisions transfer between instances under a compatible address mapping. (iv) Two decidability theorems establish total decision and determinism. (v) Three instance-non-vacuity theorems witness that the typeclass is inhabited over both abstract and concrete `EvmYul.EVM.State` values. The five concrete-state theorems extend (v) by exhibiting specific named-state instances of the trace-safety and decidability results.

18.4 The instance over `EvmYul.EVM.State`

The deposited `Instance.lean` (164 source lines, Lean 4.22.0) provides

```
instance : Parallax.EvmLikeMachine EvmYul.EVM.State
```

by wrapping six `EVMYulLean` projections:

- `step` composes `EvmYul.EVM.step` with the Cancun fork’s standard fuel and gas accounting, coercing `Except  $\tau$` to `Option` by discarding the exception payload.
- `totalSupply` folds `Account.balance` over the persistent `AccountMap`.
- `balanceOf` projects `Account.balance` at a specified address.

- `sender` reads `ExecutionEnv.sender`.
- `callDepth` reads from `Substate.accessedAccounts`. This is a conservative reading; a faithful implementation requires an upstream extension to `EVM.State` adding an explicit depth counter. The conservative reading is sound for our gate-safety theorems (it can over-approximate but not under-approximate depth), which is what they require.
- `attestationFresh` provides an adapter slot. The deposited instance uses a conservative `attestationFresh := fun _ _ => true` pending an upstream freshness primitive; the gate’s A_5 obligation is consequently vacuous on this instance, a fact explicit in the lifted theorem statements.

Method	Status	Consequence
<code>step</code>	faithful wrapper around <code>EVM.step</code>	OK
<code>totalSupply</code>	projection (fold of <code>balance</code>)	OK if balance sum is the intended notion
<code>balanceOf</code>	faithful account-balance projection	OK
<code>sender</code>	faithful environment projection	OK
<code>callDepth</code>	conservative proxy via <code>accessedAccounts</code>	A_4 backend is conservative, not exact
<code>attestationFresh</code>	currently a stub returning <code>true</code>	A_5 backend is vacuous on this instance

Table 11: EVMYulLean backend-method fidelity. The honest reporting: most methods are faithful EVM-semantics wrappers, but A_4 depth-counting and A_5 attestation freshness are not. The substrate’s lifted theorems hold as stated — the A_4 backend is sound but conservative (rejects more than necessary, never less), and the A_5 backend is vacuous (provides no A_5 assurance until an upstream freshness primitive lands).

Three levels of EVMYulLean composition. The integration’s claims separate into three levels with distinct status:

1. **Interface transport** (achieved): the abstract `EvmLikeMachine` typeclass admits an instance over `EvmYul.EVM.State` (Cancun fork). The 19 abstract theorems lift to compiled Lean 4 proof terms over this instance under a single instance registration. The deposit at [doi:10.5281/zenodo.20386868](https://doi.org/10.5281/zenodo.20386868) certifies the lifted artifact compiles byte-identically across runs.
2. **Semantic fidelity** (partial): for each obligation, the projection from `EvmYul.EVM.State` to the abstract substrate is checked above. `step`, `totalSupply`, `balanceOf`, and `sender` are faithful; `callDepth` is a conservative approximation; `attestationFresh` is a stub. Semantic fidelity is partial in the sense that two backend methods do not capture their target semantics exactly.
3. **Contract verification** (out of scope for this paper): the lifted theorems certify that any program respecting the substrate’s obligations satisfies the basis on `EvmYul.EVM.State`; they do not certify that any particular deployed bytecode respects those obligations. Verifying that a deployed contract’s execution trace satisfies the basis is the contract-verification level, and is the proper subject of downstream demos (Section 16) and downstream tooling.

A dedicated API conformance verifier (`evm_api_conformance.py`, deposited alongside the receipt) resolves 12 of 12 (100%) `EvmYul.*` identifiers referenced in `Instance.lean` against their declaration sites in EVMYulLean’s source tree at the deposited commit. Schema drift in the upstream package would surface as a build failure, an unresolved API reference, or both.

18.5 Verification protocol and reproducibility

The integration was verified on Google Colab against EVMYulLean commit 047f6307..., mathlib commit 79e94a09..., and Lean 4.22.0 (toolchain commit ba2cbbf0...). The wall-clock duration of a single run is approximately seven minutes, of which approximately three minutes is fetching the precompiled mathlib cache from the leanprover-community content-addressed distribution. The verification was executed three independent times. Each run produced byte-identical compiled artifacts for `Refinement.olean` (SHA-256 d2d9ba2f...) and `Instance.olean` (SHA-256 43c026ed...). The third compiled module, `TheoremTransfer.olean`, differs across runs only when its source file is revised; under identical source inputs it reproduces byte-identically. Byte-level reproducibility under identical declared inputs is a stronger property than recompilation alone and rules out nondeterministic compilation paths as a confounder.

The verification receipt (`receipt.json`) records the evidence chain in machine-readable form: input file SHA-256 hashes, dependency commit pins, build duration and exit code, compiled artifact fingerprints, API conformance verdict, theorem inventory, and provenance meta-data. The receipt is self-signed with SHA-256 over a canonical JSON serialization (signature prefix 9b7bb24d...) and additionally with Ed25519 against an ephemeral keypair (public key prefix 6698bfea...). A supply-chain provenance attestation (`provenance.intoto.jsonl`) conforming to the SLSA v1 schema [2] is generated alongside.

The complete artifact set—receipt, three `.olean` files, four source modules, dependency manifest, build log, conformance report, provenance attestation, two visualization figures, and offline verifier—is deposited at doi:10.5281/zenodo.20386868 [1]. The offline verifier (`verify_receipt.py`) reproduces both signature checks and the `.olean` fingerprint comparison; it depends only on the Python standard library and the `cryptography` package and does not require the Lean toolchain.

18.6 Scope

The result establishes refinement composition at the type and theorem-instantiation levels: the abstract refinement layer’s theorems hold over `EvmYul.EVM.State` modulo the typeclass-mediated translation, with proof terms compiled by the Lean 4 kernel. The result does not establish:

1. Soundness of EVMYulLean’s operational semantics. This is delegated to the upstream project’s Ethereum conformance testing (99.99 % pass rate at the deposited commit).
2. End-to-end safety of any specific deployed smart-contract bytecode. Establishing that a particular contract’s behavior under `EvmYul.EVM.step` respects the basis predicate of §6 requires producing a witness trace over that contract’s bytecode; this composition layer is treated as future work and is the natural extension above the present result.
3. A faithful implementation of A_5 oracle freshness. The deposited instance uses a conservative approximation pending an upstream extension to `EvmYul.EVM.State` adding an explicit freshness oracle field.

The substrate is therefore composable with a production EVM semantics in Lean 4; the present deposit instantiates this composition. End-to-end contract verification under the basis, for any specific contract, reduces under Theorem 43 to producing the witness trace.

19 Compositional Verification via Obligation Mapping

The substrate of §3–§6 and the EVMYulLean instance of §18 provide formal foundations. This section addresses the question that determines whether the foundation is useful in practice:

how does PARALLAX-5 compose with the existing analyzer ecosystem to produce a defensible end-to-end certificate?

19.1 Problem statement

Production smart-contract review combines outputs from multiple tools: static analyzers (Slither [7]), symbolic executors (Mythril [8], halmos [9]), proof-assistant artifacts (Certora [10], our own ObligationSol). These tools produce heterogeneous output formats with overlapping but non-identical scopes. A protocol commissioning a review typically receives a list of findings without a defensible *joint* assessment: tool A flags reentrancy, tool B reports an arithmetic concern, tool C is silent on oracles. What does the union of these reports establish? At what compliance level? With what residual gaps? Current practice answers these questions informally, through expert interpretation. We propose to formalize the answer.

19.2 Coverage as a graded function

We model each tool’s capability as a function from obligation to evidence depth on a six-level monotone ladder:

Definition 44 (Evidence depth). *The depth of evidence for an obligation is one of: 0 (no coverage), 1 (mention; code-location report, comment-grade), 2 (static-detector; pattern match flagged), 3 (symbolic-path; path-condition witness exhibited), 4 (formal-property; invariant verified by symbolic / bounded model checker), or 5 (machine-theorem; kernel-accepted theorem in an interactive prover). Depth is monotone: a depth- k claim subsumes the corresponding depth- j claim for $j \leq k$.*

Definition 45 (Tool capability). *The capability of a tool t is a function $c_t : \{A_1, \dots, A_5\} \rightarrow \{0, \dots, 5\}$ assigning each obligation the maximum depth t can in principle establish on a typical Solidity codebase under documented invocation.*

Definition 46 (Joint capability). *For a set of tools $T = \{t_1, \dots, t_n\}$, the joint capability C_T is the pointwise maximum:*

$$C_T(A_i) := \max_{t \in T} c_t(A_i).$$

The joint capability formalizes the operational claim that a multi-tool composition’s coverage is the best-of evidence available for each obligation.

19.3 Two theorems

The compositional architecture rests on two algebraic properties of joint capability. Both have simple proofs over the finite depth lattice; we state them precisely and verify them mechanically in our coordinator implementation (§19.8).

Theorem 47 (Compositional Coverage). *Let T be a set of tools with capabilities c_t and let C_T be their joint capability per Definition 46. Then for any obligation A_i and any tool t' :*

- (i) $C_T(A_i) \geq c_t(A_i)$ for all $t \in T$ (monotonicity);
- (ii) $C_{T \cup \{t'\}}(A_i) \geq C_T(A_i)$ (refinement under addition);
- (iii) $C_{T \cup \{t'\}}(A_i) = C_T(A_i)$ if and only if $c_{t'}(A_i) \leq C_T(A_i)$ (strictness).

Proof. All three claims follow from the definition of max over a finite multiset. (i) For any $t \in T$, $c_t(A_i)$ is a member of the multiset over which $C_T(A_i)$ takes the maximum, hence $C_T(A_i) \geq c_t(A_i)$. (ii) The multiset over which $C_{T \cup \{t'\}}(A_i)$ takes the maximum extends the multiset for $C_T(A_i)$ by exactly $c_{t'}(A_i)$; adding an element to a multiset cannot decrease its maximum. (iii) The maximum is unchanged by adding $c_{t'}(A_i)$ if and only if $c_{t'}(A_i)$ does not exceed the existing maximum. $\square$

Theorem 48 (Certificate Monotonicity). *Let $P : (\{A_1, \dots, A_5\} \rightarrow \{0, \dots, 5\}) \rightarrow \{0, 1, 2, 3, 4, 5\}$ be the P-level function defined by*

$$P(C) = \max\{L : C(A_i) \geq R(L) \text{ for all } A_i\}$$

where $R : \{0, \dots, 5\} \rightarrow \{0, \dots, 5\}$ is the level-to-required-depth table, monotone in L . Then for any tool set T and any additional tool t' :

$$P(C_{T \cup \{t'\}}) \geq P(C_T).$$

Proof. By Theorem 47(ii), $C_{T \cup \{t'\}}(A_i) \geq C_T(A_i)$ for every A_i . The set $\{L : C(A_i) \geq R(L) \text{ for all } A_i\}$ is a downward-closed subset of $\{0, \dots, 5\}$ (because R is monotone), and replacing C with a pointwise-greater function can only enlarge this set, never shrink it. The maximum of the enlarged set is therefore at least the maximum of the original set. $\square$

The two theorems together yield the operational claim: *adding a verification tool to the stack never lowers the joint coverage or the certifiable compliance level.*

19.4 The TOOL-MAPPING calibration artifact

Theorems 47 and 48 are content-free without empirical values for each c_t . We provide these in a separately-citable calibration artifact, **TOOL-MAPPING v1.0**, which records for each tool t and each obligation A_i the documented capability $c_t(A_i)$, justified by reference to the tool’s detector catalogue (Slither), SWC-registry coverage (Mythril), property-language expressiveness (halmos), or obligation-signature library (ObligationSol).

Table 12 reports the calibrated capability matrix for the four-tool stack. Detailed entries (33 per-finding mappings spanning the four tools) appear in the supplementary calibration document.

Tool	A_1	A_2	A_3	A_4	A_5
Slither (v0.10.x)	2	2	0	2	0
Mythril (v0.24.x)	3	3	2	3	0
halmos (v0.2.x, with property)	4	4	4	4	4
ObligationSol (PARALLAX-5 v6)	2	2	0	2	2
Joint $C_{\{S,M,h,A\}}$	4	4	4	4	4

Table 12: Calibrated capability matrix for the four-tool stack. Entries are evidence depths per Definition 44. The joint row is the pointwise maximum per Definition 46; under Theorem 48 the corresponding P-level is P_4 , provided halmos is invoked with property specifications for each obligation. The bare four-tool stack *without halmos properties* yields joint coverage (3, 3, 2, 3, 2) and P_2 .

19.5 Two observations from the matrix

The matrix establishes two empirical claims worth surfacing explicitly.

A_5 is uniquely served by ObligationSol within the static / symbolic stack. Slither has no oracle detectors; Mythril has no oracle reasoning. Without ObligationSol, the joint capability on A_5 collapses to 0 unless halmos is invoked with an oracle-staleness property. ObligationSol is therefore *necessary* (not merely useful) for any static-depth A_5 coverage. We refer to this as the *Mango Markets case* (§19.7) after the 2022 incident that exploited precisely this gap.

halmos is conditionally universal. halmos can in principle achieve depth 4 on any obligation, but only if the analyst supplies a property function asserting the invariant. Without property files, halmos’s contribution is depth 0. This asymmetry is operationally significant: the marginal cost of using halmos depends on specification effort, whereas the other tools run on raw source. The capability matrix above records halmos’s intrinsic ceiling; the coordinator records halmos’s actual contribution per analysis run.

19.6 Empirical study: the discovery and held-out corpora

We applied the coordinator to a curated 15-incident corpus drawn from the 53-incident PARALLAX-5 catalog. The corpus is split: 10 incidents for capability calibration (*discovery set*), 5 incidents held out under pre-registration (*validation set*). The discovery set spans all five obligations and the basis-observability axis (§11.4). The validation set’s predictions are sealed against the calibrated capability matrix and TOOL-MAPPING v1.0 before tool execution.

For each incident we (i) retrieve the pre-exploit contract source from Etherscan at the documented vulnerable commit; (ii) run each tool through the coordinator’s standardized runner; (iii) map findings through TOOL-MAPPING v1.0; (iv) compute the certificate with audit trail. The coordinator and runner artifact are open-source [14]; tool versions are pinned in the deposited Docker compose file.

19.7 Case study: the Mango Markets compositional case

The October 2022 Mango Markets exploit (\$115M loss) manipulated a single-source spot-price oracle to inflate collateral valuation. Per the corpus entry (*incident-009*):

- Slither reports zero findings on oracle-related code: it has no oracle detector.
- Mythril reports zero findings on oracle staleness: it has no oracle reasoning.
- halmos with no property file reports nothing.
- ObligationSol reports two findings: `A5_oracle_stale_window` (no freshness check) and `A5_oracle_quorum_insufficient` (single-source price feed).

The joint capability on A_5 for the stack $\{\text{Slither}, \text{Mythril}, \text{halmos}_{\text{cold}}\}$ is 0; the joint capability for the stack $\{\text{Slither}, \text{Mythril}, \text{halmos}_{\text{cold}}, \text{ObligationSol}\}$ is 2. By Theorem 48, adding ObligationSol cannot lower the P-level, and in this case strictly raises the per-obligation coverage on A_5 from depth 0 to depth 2 — the difference between an undefendable certificate (P_0 from missing A_5 coverage) and a defendable P_2 for any contract that does not also leave A_3 uncovered. This is what we call *compositional necessity*: a tool whose contribution is not merely additive but is the unique source of coverage on at least one obligation.

19.8 The coordinator artifact

The compositional architecture is implemented in an open-source Python package (`parallax5-coordinator`) shipped alongside this paper. The package provides:

- A capability model (`capability.py`) implementing Definitions 44–46.
- Mechanical verification of Theorems 47–48 by exhaustive check (2,152 algebraic checks, all passing) over the known-tools set and a synthetic lattice extension.
- A runner module that executes Slither, Mythril, halmos, and ObligationSol against a Solidity contract, parses output, and normalizes into the `Finding` schema.
- A certifier (`certifier.py`) that produces a versioned P-level certificate with an audit trail recording exactly which findings from which tools contribute to which obligation’s coverage at which depth.
- A command-line interface (`parallax5-coordinator`) with subcommands `analyze`, `capability`, and `mapping`.

The package is released under Apache-2.0 alongside TOOL-MAPPING v1.0; community contribution of new entries (additional Slither detectors, new tools, refined justifications) follows the standard pull-request flow. We intend the mapping table to be a living artifact updated as tool capabilities evolve.

19.9 Threats to validity

The capability matrix entries are calibrated against documented tool capabilities. Tool behaviour in production may diverge from documentation when contracts trigger detector edge cases or exhaust symbolic-execution budgets. We mitigate this by (i) pinning tool versions in the runner’s Docker image; (ii) recording per-finding mapping justifications in TOOL-MAPPING v1.0 such that recalibration is auditable; (iii) reporting both cold-run and with-property capabilities for halmos.

The held-out corpus is small ($n = 5$); larger validation is documented as a follow-on study extending to the full 53-incident catalog. We pre-register predictions to avoid hindsight calibration.

The mapping itself is a research artifact subject to community review. We invite tool vendors and adjacent researchers to challenge specific entries: a justified counterclaim refines the calibration and strengthens the joint claim.

19.10 Implication for the PARALLAX-5 standard

The compositional architecture realizes the standard’s intent (§17): PARALLAX-5 does not displace existing analyzers; it provides the obligation taxonomy under which their heterogeneous outputs compose into a single defensible compliance claim. A P-level is a function of the joint coverage; the joint coverage is a function of the tools applied; the tools are interchangeable and additive. This is the inversion that turns a fragmented tool ecosystem into a coordinated certification surface.

20 Adoption Pathways

The artifact’s commercial path is concrete, not aspirational. Five distinct constituencies have clear incentives to adopt:

Audit firms. Adopt PARALLAX-5 obligation tags ($A_1 \dots A_5$) as report headings, emit P2 or P3 certificates as audit deliverables. Differentiation: “every finding labeled by obligation; every fix verified to the labeled level.” Marginal cost: minimal (re-organizing existing reports).

Protocol teams. Publish P3+ certificates alongside audit reports. Differentiation: a single machine-readable artifact replacing dozens of PDFs. Insurer-ready. Marginal cost: ~ 1 engineer-week for typical vault per certificate; revalidation triggers reduce per-upgrade cost.

DeFi insurance underwriters. Use compliance level as a tier input. Differentiation: comparable, machine-readable risk inputs across protocols. Marginal cost: certificate-parser integration (1 engineer-week).

AI-agent platforms. Deploy the step-secure gate in front of every state-mutating action proposed by an agent. Differentiation: provable execution-time containment. Marginal cost: one wrapped contract call per agent action.

Chains and L2s. Maintain a public registry of P-level certificates per deployed contract. Differentiation: comparability for users. Marginal cost: registry contract + UI.

#	Layer	Tool	Strength
1	Bounded model checking	Z3 SMT	Witnesses + finite-depth UNSAT
2	Inductive invariants	Z3 (quantified)	Unbounded preservation
3	Basis minimality	Z3 (witnesses)	Independence of each A_i
4	Closure inhabitation	Z3 ($\times 31$)	All 31 classes realized
5	ObligationSol fixture agreement	Z3 vs regex	6/6 agreement on fixtures
6	Bytecode-level proofs	halmos	EVM symbolic execution
7	Lean 4 theorems	Lean	95 theorems, 0 <code>sorry</code>
+	BitVec(256) extension	Z3 BV mode	uint256 overflow class
+	SMT-LIB export	Z3 + standard	Solver-portable proofs

Table 13: Verification stack.

The framework does not displace existing tools, auditors, or workflows; it organizes their outputs into a single artifact that can be machine-checked, signed, and revoked.

PARALLAX-5 defines five progressive compliance levels:

- **P0 Unclassified.** No basis mapping.
- **P1 Annotated.** Each value-affecting transition mapped to $A_1 \dots A_5$ via `@axioms` declarations.
- **P2 Statically Screened.** A static analyzer produces an obligation report.
- **P3 Symbolically Checked.** Bounded counter-example search over value-affecting paths (e.g., halmos).
- **P4 Formally Proved.** SMT/Lean/Certora-style proof of every declared obligation.
- **P5 Runtime Enforced.** A step-secure gate rejects unsafe transitions and emits cryptographic certificates.

The complete schema and example certificate are in the companion document `PARALLAX-5-Standard.md`.

21 Tooling and Artifact

21.1 Theorem Inventory with Lean Names

Table 14 below maps each substantive result in this paper to its Lean 4 identifier in `Parallax5.lean`, enabling a reviewer who has the source to navigate directly to the relevant proof. The names follow a uniform convention: `snake_case` identifiers with the result’s role as the suffix (e.g., `_safe`, `_preserved`, `_sound`, `_max`). Each entry compiles in the deposited kernel run with zero `sorry`; the verification script reports the count.

21.2 Conditional ObligationSol Soundness

We make ObligationSol’s soundness conditional on parser/extractor assumptions:

Proposition 49 (Fixture agreement). *On the finite fixture corpus, ObligationSol’s verdict agrees with the SMT reference model on 6/6 cases.*

Theorem 50 (Conditional ObligationSol soundness over annotated fragment). *Assume:*

1. **Extractor completeness over $\mathcal{L}_{\text{ann}}$:** every value-affecting transition in the fragment is emitted as an obligation.
2. **Parser soundness:** every emitted obligation corresponds to the intended source-level transition.

Result	Lean name	Type
Conditional completeness	<code>conditional_completeness</code>	theorem (cond)
Conditional safety (contrap.)	<code>conditional_safety</code>	theorem (cond)
Falsification criterion	<code>completeness_iff_no_counterexample</code>	theorem
Transition-level minimality	<code>basis_minimal</code>	theorem + witnesses
σ empty iff B	<code>sigma_empty_implies_B, B_implies_sigma_empty</code>	theorem
Constructive closure inhabitation	<code>constructive_closure_inhabitation</code>	theorem
Toy-machine inhabitation	<code>toy_machine_inhabits_subset</code>	theorem
Hardened-deposit A_1 preservation	<code>a1_preserved_by_deposit_harden</code>	theorem
Hardened admin-op A_2 preservation	<code>a2_preserved_by_admin_guarded</code>	theorem
Hardened oracle-read A_5 preservation	<code>a5_preserved_by_oracle_read_guarded</code>	theorem
A_2 -completeness for unauthorized mut.	<code>a2_complete_for_unauthorized_mutations</code>	theorem
A_5 -completeness for stale oracle	<code>a5_complete_for_stale_oracle</code>	theorem
A_4 -completeness for reentrancy	<code>a4_complete_for_reentrant_mutation</code>	theorem
Compositional preservation (4 obligations)	<code>a1_compositional, ...</code>	theorems
Gate state safety	<code>basis_gate_state_safety</code>	theorem (new)
Gate transition safety	<code>basis_gate_transition_safety</code>	theorem (new)
Maximal permissive safety	<code>basis_gate_is_maximal_permissive</code>	theorem (new)
No basis-violating exploit	<code>no_basis_violating_exploit_under_sound_gate</code>	theorem (new)
Monitor soundness suffices	<code>monitor_soundness_suffices</code>	theorem
Adaptive iteration safety	<code>adaptive_iteration_preserves_security</code>	theorem
Step-secure refinement transfer	<code>step_refinement_transfer</code>	theorem (new)
On-chain indistinguishability	<code>onchain_indistinguishability_of_key_compromise</code>	theorem
Conservation wrapper preserves A_1	<code>conservation_wrapper_preserves_A1</code>	theorem
Sibling reentrancy guard preserves A_4	<code>sibling_reentrancy_guard_preserves_A4</code>	theorem
ValueBearingMachine generalization	<code>generic_agent_gate_preserves_security</code>	theorem
Cross-VM agent safety (Solana/Move/Bank)	<code>solana_agent_safe, etc.</code>	theorems

Table 14: Theorem inventory with exact Lean names. All compile without `sorry` in the Lean 4 module at `parallax/formal/lean/Parallax5.lean`.

3. **Fragment restriction** $\mathcal{L}_{\text{ann}}$: *no delegatecall, no inline assembly, no dynamic dispatch through unknown externals, no upgradeable storage collision, no hidden callbacks, all value-affecting entrypoints visible, all external data reads annotated.*

Then if *ObligationSol* accepts the contract, the generated obligation set implies $A_1, \dots, A_5$ for all analyzed transitions.

Proposition 51 (ObligationSol incompleteness). *There exists a contract outside $\mathcal{L}_{\text{ann}}$ that violates A_4 but is accepted by ObligationSol. The Solv-clone with sibling-callback cross-function reentrancy is the canonical witness.*

22 The Certificate Schema

The coordinator’s output, referenced throughout this paper as “the certificate,” is specified by a machine-readable schema released as RFC v1.0 (deposit `docs/CERTIFICATE.SCHEMA.md`, with JSON Schema realization at `schemas/certificate_v1.json` validated as Draft 2020-12). This section gives the schema’s formal content: its field-level specification, its lifecycle state machine, its canonical serialization for deterministic fingerprinting, and the conformance requirements it imposes on issuers, validators, and consumers.

22.1 Top-level fields

A v1.0 certificate is a JSON object containing exactly the following 19 top-level fields:

Field	Type	Purpose
schema_version	string (semver)	Schema version tag (currently "1.0.0")
certificate_id	uuid v4	Globally unique identifier
issued_at	ISO 8601	Issuance timestamp
validity	object	not_before, not_after, lifecycle_state
issuer	object	Issuer name and public key
protocol	object	Name, version, identifier, category
contract	object	Source hash, address, chain id
trust_base	object	Cryptographic and refinement assumptions
obligation_coverage	object	Depth assignment for each of A1–A5
evidence	array	List of evidence entries (tool findings, theorems)
crops_vector	object	Computed CROPS rating (C, R, O, P, S)
walkaway	object	Classification, proof reference, dependencies
mapping	object	Tool-mapping reference (namespace, version, DOI)
fingerprint	sha-256 hex	Canonical-serialization hash
signature	object	Ed25519 signature over fingerprint
supersedes	uuid (optional)	Prior certificate this revision replaces
revocation_reason	string (optional)	Populated when lifecycle state is revoked
registry_anchor	object (optional)	On-chain anchor (tx hash, block) once registered
forks	array (optional)	References to derivative forks

Worked-example excerpt. A concrete certificate (for the patched ERC-4626 vault of §16.1) renders as:

```
{
  "schema_version": "1.0.0",
  "certificate_id": "fa52e2f3-9d4e-4dad-b5f5-ac78dc170c31",
  "issued_at": "2026-05-26T11:43:00Z",
  "validity": { "not_before": "2026-05-26", "lifecycle_state": "issued" },
  "issuer": { "name": "AquaUrsa Research", "public_key": "ed25519:6698bfea..." },
  "protocol": { "name": "PARALLAX-5 Demo 1 - PatchedVault",
    "category": "vault", "identifier": "demos/vault" },
  "contract": { "source_hash": "sha256:7a304cbe...", "chain_id": null },
  "obligation_coverage": {
    "A1": { "depth": 4, "evidence_refs": ["ev-lean-001"] },
    "A2": { "depth": 4, "evidence_refs": ["ev-lean-002"] },
    "A3": { "depth": 0, "evidence_refs": [] },
    "A4": { "depth": 2, "evidence_refs": ["ev-slither-001"] },
    "A5": { "depth": 0, "evidence_refs": [] }
  },
  "evidence": [
    { "evidence_id": "ev-lean-001", "tool": "lean4_kernel",
      "obligation_mapped_to": "A1", "depth_contribution": 4,
      "justification": "Conservation.lean: deposit_preserves_conservaion, ..." },
    { "evidence_id": "ev-slither-001", "tool": "slither",
      "obligation_mapped_to": "A4", "depth_contribution": 2,
      "justification": "reentrancy-no-eth (Medium)" }
  ],
  "crops_vector": { "C": 4, "R": 5, "O": 5, "P": 0, "S": 4,
    "computation_method": "max_within_dimension" },
  "walkaway": { "classification": "full",
```

```

    "proof_ref": "demos/vault/proof/Conservation.lean",
    "dependencies_disclosed": [] },
"mapping": { "namespace": "tool-mapping/aquaursa-v1",
             "version": "1.0.0", "doi": "10.5281/zenodo.20386868" },
"fingerprint": "
    a880650b924463c61c78c014f4966e554fa59913941e872e036255259a8da86d",
"signature": { "algorithm": "ed25519", "message": "<fingerprint>",
               "signature": "<64 bytes hex>" }
}

```

The excerpt elides the `registry-anchor`, `supersedes`, and `fork-reference` fields (all optional). The full schema is in `schemas/certificate_v1.json`; the worked example is in `examples/certificate_uniswap_v3_core.json` (a different protocol, but the structure is identical).

22.2 Lifecycle state machine

A certificate occupies exactly one of seven lifecycle states at any time. Transitions between states are restricted as follows:

Definition 52 (Certificate lifecycle). *The state space is*

$$\{\text{draft}, \text{issued}, \text{published}, \text{superseded}, \text{revoked}, \text{expired}, \text{withdrawn}\}$$

with admissible transitions:

$$\text{draft} \rightarrow \text{issued} \rightarrow \text{published} \rightarrow \{\text{superseded}, \text{revoked}, \text{expired}, \text{withdrawn}\}.$$

The terminal states are absorbing. The transition from issued to published is the registry-anchor commit; the transition to superseded requires a successor certificate naming the predecessor via supersedes.

22.3 Canonical serialization and fingerprint

Definition 53 (Canonical serialization). *Let J be a certificate object with the **fingerprint** and **signature** fields removed. The canonical serialization $\text{Canon}(J)$ is the lexicographically-key-sorted JSON encoding of J , with no whitespace except after ":" and "," (one space each), with UTF-8 encoding, with all string literals normalized to NFC, and with no trailing newline.*

Definition 54 (Fingerprint). *The fingerprint of a certificate is $\text{SHA-256}(\text{Canon}(J))$, where J is the certificate object with the fingerprint and signature fields removed.*

The fingerprint is the input to the Ed25519 signature, and the signature's `message` field is exactly the fingerprint byte string. This construction has the property that any modification to any other field changes the fingerprint, which is the substrate's tamper-detection primitive. The validator command `parallax5 validate` recomputes the fingerprint from the post-removal canonical serialization and compares it to the stored fingerprint; mismatch indicates tampering.

22.4 Evidence resolution

Definition 55 (Evidence resolution validity). *A certificate C has valid evidence resolution iff for each obligation A_i in `obligation_coverage`, its depth d_i satisfies $d_i = \max_{e \in C.\text{evidence}: e.\text{obligation} = A_i} e.\text{depth_contribution}$.*

This is the operational form of compositional coverage (Theorem 47): the certificate's per-obligation depth is determined by the strongest piece of evidence in its catalog. The validator enforces this constraint and rejects certificates whose claimed depth exceeds what its evidence supports.

22.5 Registry events

The certificate’s lifecycle is anchored on chain via the `ParallaxRegistry` contract at `registry/src/ParallaxRegistry.sol` (Apache-2.0, Solidity 0.8.24, deployable to any EVM chain). A reference deployment is live on Sepolia at `0x8015A98dF9037Cd79a03B291a6fF3C2841992D5b` (source verified on Etherscan), with the first certificate already anchored under the canonical vault-demo fingerprint. The contract is permissionless: any party may call `issue(bytes32 fingerprint)` to register a new certificate; only the original registrant may transition that record through the schema’s lifecycle states. Six event kinds are emitted—`Issued`, `Published`, `Superseded`, `Revoked`, `Expired`, `Withdrawn`—each carrying the certificate’s fingerprint and the relevant transition metadata, with the fingerprint and issuer indexed for efficient query. Consumers index the registry by fingerprint to retrieve the currently-effective certificate, or by issuer address to enumerate an issuer’s complete certification history.

The contract’s gas profile (verified against `anvil`) is: `issue()` 115k, `publish()` 32k, `supersede()` 38k, `revoke()` 38k, `expire()` 32k, `withdraw()` 32k. At mainnet pricing of 30 gwei and \$2,500/ETH, an `issue()` call costs approximately \$8.60; subsequent state transitions cost approximately \$2.40 each.

State-machine soundness. The contract’s lifecycle state machine is formally verified in `lean/Parallax5/Registry.lean` (6 theorems, 0 sorry). The proved properties are: (i) terminal states absorb (no transition originates from `Superseded`, `Revoked`, `Expired`, or `Withdrawn`); (ii) the view function `isEffective()` returns true exactly for the operational states `{Issued, Published}`; (iii) every transition into a terminal state originates from `Published`; (iv) the four targets reachable from `Published` are exactly the four terminal states; (v) the contract’s `SelfSupersession` revert proves a record cannot supersede itself (the supersession edge forms a strict partial order); (vi) all six transitions preserve the `ValidState` invariant. These theorems together establish that the on-chain state machine implements the schema RFC v1.0 §3 specification.

Self-application. The registry contract is itself a value-bearing on-chain artifact and can therefore carry its own PARALLAX-5 certificate. A canonical self-certificate records: obligation coverage at D_4 for A_1 (state integrity, proved by `Registry.lean`) and A_2 (authorization closure, proved by the same module); walkaway class *full* (no admin role, no pausing mechanism, no upgrade path); CROPS vector (4, 5, 5, 0, 4). This self-application demonstrates substrate maturity: the infrastructure that records certificates carries a certificate that satisfies the same obligations it records.

Test coverage. The contract is covered by 24 Foundry tests at `registry/test/ParallaxRegistry.t.sol`, including 2 fuzz tests at 256 runs each, exercising every state transition, every revert path, and the absorption invariance of all four terminal states. All 24 pass.

22.6 Conformance requirements

- **Issuers** MUST produce certificates that validate against `schemas/certificate_v1.json` as Draft 2020-12; MUST sign the fingerprint with a public key declared in the `issuer` object; MUST publish the public key out of band (e.g., DOI, well-known URL).
- **Validators** MUST reject any certificate whose computed fingerprint does not match its stored fingerprint; MUST reject any certificate whose evidence resolution is invalid; MUST reject any certificate signed with a key not associated with the named issuer.
- **Consumers** SHOULD evaluate the certificate’s CROPS vector against an explicit policy, and SHOULD NOT rely on the certificate as a substitute for the policy.

A worked example (`examples/certificate_uniswap_v3_core.json`) validates cleanly under `parallax5 validate` and produces the CROPS vector reproduced in Section 15.

23 Substrate Governance

A coordination layer’s value depends on its credibility as a coordination layer. A coordination layer captured by a single party — whether commercial or state — ceases to coordinate the field. PARALLAX-5 addresses this risk structurally: by placing the standard text under a license whose effects cannot be revoked, by foregoing trademark and patent claims, and by specifying the fork procedure under which derivative standards can co-exist without fragmentation.

23.1 The Non-Capturability Charter

The Charter (deposit `NON_CAPTURABILITY_CHARTER.md`) is an eleven-article governance commitment under CC0 (Creative Commons Zero). The articles state the substrate’s commitments structurally rather than aspirationally:

- Article 1 declares the substrate’s standard text under CC0; no commitment to keep the text under CC0 is needed because the license is irrevocable by construction.
- Article 2 declares structural irrevocability: AquaUrsa Research makes no patent claims over techniques disclosed in the substrate and does not intend to seek such protection; this is a statement about AquaUrsa’s own conduct, not a warranty regarding third-party patent rights. No trademark is asserted on the name PARALLAX-5; no token is issued; no governance body is established; no treasury is established.
- Article 3 declares walkaway commitment for the substrate itself: there is no role within the substrate whose disappearance would impede the substrate’s continued use by anyone.
- Article 4 prohibits any party (including the original author) from establishing exclusive control over substrate-derivative products. Commercial layers built on top of the substrate are permitted; capture of the substrate itself is foreclosed.
- Articles 5–10 specify the corresponding details for credentialing, certification, the registry, the tool mapping, and the certificate schema.
- Article 11 specifies that the Charter’s amendment protocol is a fixed point under itself: any amendment that would weaken Articles 1–10 is automatically void.

The Charter is structurally enforceable rather than reputationally enforceable. CC0 cannot be revoked. A patent that was not filed cannot retroactively encumber the technique. A trademark that was never asserted cannot be enforced. A governance body that was never established cannot vote. This is the substance of “structural irrevocability.”

23.2 License Layering

The substrate distinguishes the public coordination layer from implementation and service layers. The PARALLAX-5 standard text — including the obligation vocabulary (A_1 – A_5), the depth scale (D_0 – D_5), the PARALLAX-CROPS dimensions, the walkaway taxonomy, the certificate field semantics, and this paper’s specification content — is dedicated under CC0 1.0 Universal. Reference software implementations, including the coordinator (`parallax5_coordinator`), the validator, the certificate registry contract (`ParallaxRegistry.sol`), the CLI, and the Lean and SDK artifacts, are released under Apache-2.0 unless an individual file declares otherwise. The paper itself is licensed under CC-BY 4.0 to preserve academic attribution norms. AquaUrsa Research retains control over its own issuer keys, hosted services, support offerings, signed certificate attestations, and commercial validation processes; this control does not extend to the public standard vocabulary.

The structural consequence: a third party may freely adopt PARALLAX-5 terminology, issue PARALLAX-5 certificates, deploy the registry contract, operate a registry instance, and build commercial services on the substrate without permission from AquaUrsa Research. A protocol team relying on a specific issuer’s attestation distinguishes between the public substrate and the issuer’s signed identity at the certificate level: each certificate carries an issuer public key, a fingerprint, and a registry anchor, and the substrate’s verification logic treats these as the basis of attestation trust. The substrate’s reputation depends on the cryptographic integrity of these mechanisms, not on trademark enforcement.

23.3 The Fork Protocol

The Fork Protocol (deposit `FORK_PROTOCOL.md`) addresses the practical question of standard evolution: how can a derivative version of PARALLAX-5 co-exist with the canonical version without fragmenting the field’s vocabulary?

Definition 56 (Fork classification). *A derivative standard is classified into exactly one of four fork types:*

- **Calibration fork** — *adjusts the depth assignments in the tool-mapping without changing the substrate’s vocabulary; co-exists with the canonical via parameter overrides.*
- **Extension fork** — *adds new obligation classes or new evidence-source kinds; specifies a default null contribution to existing dimensions.*
- **Refinement fork** — *replaces some obligation definitions with stricter variants; provides a forward map from canonical evidence to refined evidence.*
- **Independent reformulation** — *the derivative replaces substantial portions of the substrate and is not jointly composable with the canonical; classified to make this clear to consumers.*

Definition 57 (Compatibility levels). *A fork is further classified by its compatibility relation to the canonical: strict (certificates from one validate under the other), additive (canonical certificates lift to fork without modification, but not the reverse), semantic-divergence (certificates do not interchange but the field vocabulary is mutually recognizable), or independent (separate vocabularies).*

A pull-back procedure specifies how a fork’s improvements can be merged into the canonical via standard governance (no governance body required; the merge is procedural and effected by the original author or any successor maintainer publishing the merged version). The Fork Protocol’s existence is itself an anti-capture mechanism: if the canonical’s stewardship becomes contentious, an independent fork can succeed without the field’s vocabulary fragmenting.

23.4 Why CC0 is structurally necessary

A commercial substrate would fail at coordination because adoption would mean accepting a licensing cost that grows with the substrate’s success. The substrate’s value derives from being a shared vocabulary across protocols; a shared vocabulary cannot be priced because pricing impedes sharing. CC0 is the only license that admits both unlimited commercial use *and* unlimited modification with no attribution requirement. The choice is not values-driven but mechanism-driven: CC0 is the license that makes the substrate work.

This is more than a slogan. Consider the failure modes the choice forecloses. A copy-left license (GPL family) would require derivative works to be similarly licensed, which would interfere with proprietary commercial layers built on top of the substrate. A permissive-with-attribution license (MIT, BSD, Apache-2.0) would require every certificate, every audit report, every wallet UI that displays a CROPS vector to include attribution, which would either be ignored (defeating the license) or comply (creating friction). A no-derivatives clause would freeze

the substrate at its initial state. A non-commercial clause would block the very commercial layers that drive adoption. CC0 is the unique license whose semantics match the coordination function the substrate is designed to provide.

The substrate enables commercial products built on top of it (see §20: the registry, AI-FV co-pilot, agent runtime gate deployment as a service), but the substrate itself is non-commercial by construction. The Charter’s Article 4 prohibits any party — including the original author — from establishing exclusive control over substrate-derivative products. Commercial layers compete on execution quality; the substrate underlying them remains common.

24 Claims Table

Claim	Type	Evidence
Conditional completeness (assuming adequacy)	theorem	Lean
Adequacy of basis for on-chain code bugs	assumption + empirical	catalog + halmos
Adequacy for off-chain-rooted basis-observable consequences	assumption + empirical	Resolv / Drift / Kelp
Step-secure gate is maximally permissive	theorem	Lean
Monitor soundness suffices for safety	theorem	Lean
Adaptive policies cannot escape envelope	theorem	Lean
Cross-VM transfer (Solana / Move / banking)	theorem (under step refinement)	Lean
EVMYulLean refinement (composed instance verified)	theorem (instance-witnessed)	Zenodo deposit
Compositional coverage & monotonicity	theorem (lattice-exhaustive)	coordinator + TOOL-MAPPING v1.0
TOOL-MAPPING v1.0 calibration	calibration artifact + justifications	schemas/tool_mapping_v1.json
On-chain indistinguishability	theorem	Lean
$p^* = (1 - c/v)/(1 - \epsilon)$	proposition (game theory)	math + Z3
$L_{\text{basis-observable}} = \4.01B	empirical	catalog.csv
$L_{\text{basis-unobservable}} = \1.62B	empirical	catalog.csv
95 Lean theorems, 0 sorry	artifact	Parallax5.lean
129 Python fire tests (9 suites)	artifact	fire_tests.py

Table 15: Claims table separating theorems, assumptions, empirical evidence, and artifact claims.

25 Related Work

Smart-contract static analysis. Slither [7] pioneered syntactic pattern matching; its detector library overlaps with our obligation patterns but is presented as a checklist. Mythril [8] and halmos [9] provide symbolic execution. None of these tools claim or prove a basis-completeness property.

Formal verification. Certora [10] verifies user-supplied rules via SMT; KEVM [11] provides full EVM semantics in K. Our work is complementary: we identify a small fixed obligation set covering the empirically observed vulnerability space, verifiable by any of the above tools.

Capability-based and noninterference security. The general approach of conservation-law-like invariants has antecedents in noninterference [12]. Our novelty: the specific basis, the conditional-completeness formulation, the two-axis empirical classification, and the cross-domain generalization.

AI agent safety. Constitutional AI [13] addresses agent behavior at training time. Our framework addresses execution time via a step-secure gate. The two are complementary.

26 Limitations

The basis is empirically validated for on-chain consequence transitions in our 2016–2026 corpus. We do not prove semantic completeness against the universe of EVM programs. The conditional-completeness theorem is conditional on the adequacy assumption; the empirical corpus and halmos reproductions support adequacy, but adequacy is itself falsifiable.

ObligationSol misses cross-function reentrancy; the substrate composes ObligationSol with halmos to bridge the incompleteness gap.

The Lean module covers A_1, A_2, A_4, A_5 with full proofs; A_3 is verified at the SMT and bytecode level. The cryptographic assumption (ECDSA EUF-CMA) is external to the Lean formalization.

The basis-observability classification of some incidents is necessarily judgment-dependent. Drift in particular sits on a boundary that depends on monitor design. We mark such cases as *ambiguous* rather than forcing a binary choice.

The framework’s central claim is best tested by adversarial review. We invite the community to attempt falsification: find a basis counterexample — a trust-base-respecting loss-inducing transition that satisfies all five obligations.

27 Conclusion

We have proposed PARALLAX-5: a five-obligation transition interface, a parameterized observability boundary, a step-secure execution-time shield, and a typeclass refinement to a production EVM semantics. Where the basis is adequate and monitors are sound, the substrate gives a conditionally complete execution-time shield for value-bearing transitions. Where loss arises from information absent from the on-chain trace, the indistinguishability theorem identifies the irreducible scope boundary and the need for complementary trust-base controls. Mixed incidents are handled by separating root cause from basis-observable transition consequence.

The ecosystem layer makes four additions to the formal core. The walkaway property (§12) replaces “decentralization” — a slogan — with a falsifiable predicate over protocol artifacts and a five-level classification verified by inspection of obligation traces under admin quiescence. The PARALLAX-CROPS vector (§15) extends the single-axis depth scale to a five-dimensional rating that captures dimensions of trust beyond security alone, enabling consumer-side policy filtering without bespoke per-protocol diligence; the v1.0.1 contribution matrix is conservative by design, refusing to inflate privacy ratings from incidental signature-integrity evidence. The three demonstrations (§16) exhibit the substrate working end-to-end across three protocol categories — vault, bridge, agent runtime — each composing heterogeneous evidence (static analysis, symbolic execution, Lean proof, runtime enforcement) under a single fingerprinted certificate. The certificate schema (§22) makes that certificate machine-readable, machine-validatable, and on-chain-anchorable.

The governance commitments (§23) establish that the substrate is structurally uncapturable: not by promise but by the absence of the levers through which capture would proceed. CC0 cannot be revoked. A patent that was not filed cannot retroactively encumber. A trademark that was never asserted cannot be enforced. A governance body that was never established cannot vote. These commitments are mechanism, not aspiration.

The substrate’s natural application extends beyond DeFi to any setting where autonomous agents propose state-mutating actions against value-bearing ledgers. The three demonstrations together cover (i) protocols where the substrate’s value is composing static and formal evidence, (ii) protocols where the substrate’s value is honest reporting of dependencies, and (iii) protocols where the substrate’s value is runtime enforcement itself. We expect the third class to grow most rapidly as autonomous-agent deployment expands; the AI-Agent Containment Theorem (Theorem 42) provides the formal guarantee that makes that growth safe.

The substrate’s central claim remains falsifiable: if a reader can produce a trust-base-respecting, loss-inducing transition that satisfies all five obligations, the claim is refuted, and the substrate’s vocabulary requires extension or repair. The catalog reports zero such refutations over 53 historical incidents and 13 forward-tested 2026 incidents. That is the substantive guarantee. The remainder of the substrate — the schema, the lifecycle, the CROPS vector, the registry, the agent gate — is infrastructure that becomes meaningful in proportion to the strength of that guarantee.

References

- [1] B. P. Duncan, *PARALLAX-5 / EVMYulLean Integration Verification: Machine-Checked Composition of an Abstract Refinement Substrate with a Production EVM Semantics in Lean 4*, Version 1.0.0. Zenodo, May 2026. doi:10.5281/zenodo.20386868. Verification identifier pax5-evmyul-2026-05-25-9b7bb24d91c5; receipt SHA-256 9b7bb24d91c58e98...; externally verifiable via included `verify_receipt.py`.
- [2] Open Source Security Foundation, *Supply-chain Levels for Software Artifacts (SLSA) Specification, Version 1.0*, 2023. <https://slsa.dev/spec/v1.0/>
- [3] Rekt News, *Leaderboard of public DeFi exploit losses*, 2024–2026. <https://rekt.news/leaderboard/>
- [4] DefiLlama, *DeFi total value locked dashboard and historical incident data*. <https://defillama.com/>
- [5] OWASP Foundation, *Smart Contract Top 10 (2025 Release)*, 2025. <https://owasp.org/www-project-smart-contract-top-10/>
- [6] Halborn, *Top 100 DeFi Hacks Across Multiple Networks: Cumulative Loss Analysis*, technical report, 2024.
- [7] J. Feist, G. Grieco, and A. Groce, *Slither: A Static Analysis Framework For Smart Contracts*, in Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), 2019.
- [8] B. Mueller, *Mythril: A security analysis tool for Ethereum smart contracts*. ConsenSys Diligence, 2018–2026. <https://github.com/ConsenSys/mythril>
- [9] a16z Crypto Research, *Halmos: A symbolic bounded model checker for the EVM*. <https://github.com/a16z/halmos>
- [10] Certora, *The Certora Prover*, formal verification platform documentation, 2020–2026. <https://www.certora.com/>
- [11] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Roşu, *KEVM: A Complete Formal Semantics of the Ethereum Virtual Machine*, in 2018 IEEE 31st Computer Security Foundations Symposium (CSF), pp. 204–217, 2018. Project repository: <https://github.com/runtimeverification/evm-semantics>.
- [12] J. A. Goguen and J. Meseguer, *Security Policies and Security Models*, in Proceedings of the 1982 IEEE Symposium on Security and Privacy, pp. 11–20, 1982.
- [13] Y. Bai et al., *Constitutional AI: Harmlessness from AI Feedback*, technical report, Anthropic, 2022. <https://arxiv.org/abs/2212.08073>
- [14] B. P. Duncan, *parallax5_coordinator: Compositional smart-contract verification via PARALLAX-5 obligation mapping*, 2026. Open-source Python package; ships with TOOL-MAPPING v1.0 calibration artifact.

A A Quantitative Risk Framework

The PARALLAX-5 compliance ladder admits a principled actuarial framework whose detailed calibration, market-sizing analysis, and per-protocol premium estimates are outside the scope of this academic paper and are deferred to a companion document. We give here only the framework sketch sufficient to demonstrate that the certificate’s CROPS vector and obligation-depth assignment carry information useful for underwriting, without taking positions on actuarial assumptions or naming specific protocols.

Let r_b be a baseline annual incident rate per dollar of TVL (empirically calibrable on the catalog), and let $c_L \in [0, 1]$ be the coverage fraction at compliance level L — the fraction of basis-observable losses prevented in expectation at that level. Then the expected annual loss for a protocol with TVL V and compliance level L is approximately

$$\mathbb{E}[\text{Loss}_L] \approx r_b V \left[f_{\Omega_{\text{chain}}} (1 - c_L) + f_{\Omega_{\text{config}}} (1 - c_L^{\text{cfg}}) + f_{\text{irreducible}} \right]$$

with $(f_{\Omega_{\text{chain}}}, f_{\Omega_{\text{config}}}, f_{\text{irreducible}}) = (0.603, 0.229, 0.167)$ from the unified loss ledger (Table 6). The model structure is informative: compliance at $L \geq P_3$ reduces expected loss roughly by the chain-observable share weighted by coverage; the irreducible share is unaffected by compliance level and provides the residual loss floor.

Archetype	Indicative level	Reasoning
Large lending protocol	P_2	A_1, A_2 verified; A_5 oracle dependency
AMM core (constant-product)	P_3	A_1 invariant verified; no external oracles
Liquid staking protocol	P_2	A_5 (oracle / operator quorum) is primary dependency
Stablecoin / CDP system	P_3	A_1 solvency verified; A_5 quorum on oracles
Lending protocol (modular markets)	P_3	A_1 per-market; A_5 oracle adapters
Greenfield, no prior audit	P_0 – P_1	no obligation coverage; static analysis only
PARALLAX-deployed protocol	P_4 – P_5	basis-complete static coverage with formal proof

Table 16: Archetype-level indicative compliance assignments. These are framework-level archetypes, not protocol ratings; concrete protocol-specific assessments require engagement with the protocol’s audit history, present-day code, and operational posture. The PARALLAX-5 paper takes no position on the present-day compliance level of any specific named protocol; doing so requires the protocol’s participation in a certificate issuance process and is the proper subject of underwriting practice rather than academic prose.

The certificate’s CROPS vector and obligation-depth assignment supply the inputs an underwriter needs to evaluate c_L at the protocol level; the framework is parameterized so that insurers and risk teams can substitute their own loss-history priors. Market sizing, take-rate analysis, and premium calibration against any specific protocol are deferred to a separate business document and are not endorsed by this paper. The framework establishes only that the substrate’s outputs are sufficient to support such analysis when undertaken with the appropriate actuarial discipline.

DOI identity. The DOI 10.5281/zenodo.20386868 refers to the *verification artifact* (the EVMYulLean integration Lean kernel run with the receipt and Ed25519 attestation), not to this paper. This artifact is the deposit on whose existence the abstract’s interface-transport claim depends. The paper itself, when deposited to Zenodo, will receive its own DOI; the paper deposit will cite the verification-artifact DOI as a related identifier. Readers seeking the

verification artifact should retrieve it from Zenodo at the cited DOI; readers seeking the paper or the substrate’s source repository should consult <https://github.com/aquaurisa/parallax-5>.

The complete artifact—Z3 models, halmos test contracts, Lean 4 module, SMT-LIB exports, empirical catalog with sources and confidence labels (`paper/supplement/catalog.csv`), three demos, PARALLAX-5 reference CLI validator, JSON Schema, example certificate, and verification recipe `RUN_VERIFICATION.sh`—is available at the public repository <https://github.com/aquaurisa/parallax-5>. All 95 Lean theorems compile clean without `sorry`; all 129 Python fire tests pass; all halmos symbolic-execution verdicts reproduce; all 19 CROPS tests pass; the 2,152 compositional theorem checks pass; the CLI validator accepts the canonical example and rejects deliberately-broken certificates; the three demos run end-to-end via `make demo-all`; on commodity hardware the full self-test completes in under 30 seconds. A clean-room reproducibility log is included with the deposit as `REPRODUCIBILITY_LOG.txt`.

B Codebook Inter-Rater Agreement

As a disciplined internal lower bound, we implement a codebook-driven inter-rater agreement harness at `parallax/formal/inter_rater.py` and report Cohen’s κ from two independent automated reasoning orders on the 53-incident catalog:

- Classifier A reasons root-cause first, then infers observability.
- Classifier B reasons obligation-signature first, then infers observability.

These are codebook-faithful but independent: A starts from the `root_cause_class` field and applies decision rules; B starts from the `axiom_violations` field and applies different decision rules.

Comparison	Cohen’s κ	Percent agreement
A vs B (independent reasoning orders)	0.334 (fair)	73.6%
A vs catalog (codebook recovery)	0.582 (moderate)	86.8%
B vs catalog (codebook recovery)	0.579 (moderate)	83.0%

Table 17: Codebook-driven inter-rater agreement on the 53-incident catalog. The two automated classifiers achieve *fair* agreement with each other ($\kappa = 0.334$) and *moderate* agreement against the catalog. The 14 disagreements between A and B are concentrated in the *ambiguous* category, where the codebook itself admits multiple defensible classifications — a genuine empirical finding that we surface rather than hide.

This is *not* a substitute for human inter-rater validation, which we identify as the next external step. It is a disciplined internal lower bound, more precisely framed as *internal codebook recoverability*: the catalog is roughly 85% recoverable from codebook rules alone, with the residual disagreement concentrated in cases where the rater’s choice of reasoning order matters.

Concrete plan for external human classification review. We commit to the following procedure. (i) Recruit two independent classifiers with prior smart-contract security experience (e.g., audit-firm associates, academic researchers), blinded to each other’s outputs and to the catalog’s existing classifications. (ii) Each classifier independently reclassifies a stratified random sample of $n = 25$ incidents drawn from the catalog, using only `paper/CLASSIFICATION_CODEBOOK.md` as their procedure. (iii) Compute Cohen’s κ between the two human classifiers, between each human and the catalog, and between the human pair and the automated reasoning-order pair (Classifier A / Classifier B above). (iv) Pre-register the expected agreement thresholds: $\kappa_{\text{human-human}} \geq 0.6$ (substantial) is a success criterion, $0.4 \leq \kappa < 0.6$ (moderate) requires codebook refinement, and $\kappa < 0.4$ falsifies the codebook’s usability claim. (v) Publish the per-incident classifications and full Cohen’s κ analyses, including per-axis breakdowns (root cause; basis-observability; observation-set assignment). The protocol is documented at `paper/INTER_RATER_PROTOCOL.md`; the recruiting and execution are part of the substrate’s open follow-on work.

C Protocol Archetypes

To validate framework operability on real systems without protocol-team engagement, we apply PARALLAX-5 to five protocol *archetypes* representative of the DeFi landscape. These are not endorsements or formal ratings of specific named protocols; they are pedagogical mappings illustrating the per-archetype obligation structure. Concrete named applications are deferred to the business companion document for protocols that opt in.

Archetype	Indicative level	Distinguishing obligations
Large lending protocol	P2	A1, A2 verified; A5 oracle dependency
AMM core (constant-product)	P3	A1 invariant Certora-verified, no external oracles
Liquid staking protocol	P2	A5 (oracle / operator quorum) is primary dependency
Stablecoin / CDP system	P3	A1 solvency verified; A5 quorum on oracles
Lending protocol (modular markets)	P3	A1 per-market; A5 oracle adapters

Table 18: Archetype P-level mappings. These illustrate the framework’s per-archetype obligation structure; they are not specific protocol ratings. Each archetype’s emphasized obligations should drive its certificate’s `proof_artifacts` section. Five archetype certificate sketches (legacy v8-era schema) are at `paper/supplement/real_protocols/*.json`; these document the framework’s per-archetype obligation structure but predate Certificate Schema v1.0 and do not validate against it. v1.0-validating examples are at `examples/certificate_uniswap_v3_core.json`, with additional examples at `demos/*/output/certificate.json`. Concrete v1.0-conforming named-protocol assessments require protocol-team engagement and are deferred to the business companion document.

The archetype catalog illustrates that the framework is operational on real systems, not just toy examples. The exercise surfaces the natural revalidation cadence (90–180 days, faster on governance upgrades) and the per-archetype set of trust-base controls.

D Artifact Map

The full artifact map is published as `ARTIFACT_MAP.md` (Markdown) and `artifact_map.json` (machine-readable) in the repository root. The table below summarizes the artifact groups with their headline counts and the verification command that establishes each group’s central claim. Per-file detail is in the supplements.

Artifact group	Headline counts	Verification command
Formal core (v8 carryover)	95 Lean theorems, 0 <code>sorry</code> ; 129 fire tests; 53-incident catalog; 4 halmos archetype pairs	<code>./parallax/formal/lean/verify.sh</code>
Substrate foundations	11-article Charter; 4-type Fork Protocol; 19-field schema; 6 Walkaway theorems; CROPS v1.0.1 (19 tests)	<code>python -m parallax5.coordinator.theorems;</code> <code>python tests/test_crops.py</code>
Worked examples	3 demos (vault, bridge, agent gate); 4 Lean files, 21 theorems, 0 <code>sorry</code> combined; 3 validated certificates	<code>make demo-all</code>
Onchain certificate registry	<code>ParallaxRegistry.sol</code> (Apache-2.0); 24 Foundry tests pass; <code>Registry.lean</code> 6 theorems 0 <code>sorry</code> ; Python client + CLI integration	<code>cd registry && forge test;</code> <code>./scripts/deploy.sh anvil</code>
Coordinator and CLI	12 unified subcommands (6 v9 + 6 v8); 2,152 compositional checks	<code>parallax5 --help;</code> <code>python -m parallax5.coordinator.theorems</code>
Paper (this document)	46 pages; 25 theorem labels, 19 definition labels	<code>cd paper && pdflatex parallax-5_v9_2.tex</code>
Full pipeline	all of the above	<code>./RUN_VERIFICATION.sh</code>

Table 19: Artifact-group summary. The detailed per-file map (file paths, Lean line counts, certificate fingerprints, halmos witness inputs) is in `ARTIFACT_MAP.md` at the repository root, with a machine-readable JSON variant at `artifact_map.json`. Splitting the detail out of the PDF keeps both representations renderable without clipping and ensures the machine-readable form is canonical.